

Scientific Computing

Aditya Dendukuri

UC Santa Barbara

Contents

Preface	5
1 Fundamentals	6
1.1 Mathematical Notation and Proof Techniques	6
1.1.1 Logic and Set Notation	6
1.1.2 Proof Techniques	7
1.1.3 Notation Conventions	7
1.2 Vectors and Matrices	8
1.3 Matrix Arithmetic	12
1.3.1 Addition and Subtraction	12
1.3.2 Dot Product	14
1.3.3 Matrix Inverse	17
1.4 Introduction to Norms	19
1.5 Computational Complexity	20
1.5.1 Big-O Notation	21
1.5.2 Flop Counting	21
1.6 Floating Point Arithmetic	22
1.6.1 Floating Point Representation	22
1.6.2 IEEE 754 Special Values	23
1.6.3 Arithmetic Pitfalls	24
1.7 Quantifying Approximation Error	24
1.7.1 Absolute and Relative Errors	24
1.7.2 Error Analysis Frameworks	25
1.8 Python and NumPy Primer	25
1.8.1 Arrays and Basic Operations	25
1.8.2 Linear Algebra Operations	26
1.8.3 Key Idioms	26

2	Direct Methods for Linear Systems	27
2.1	Linearity	27
2.1.1	Linear Systems	27
2.1.2	Invertibility and Existence	27
2.1.3	Linear Combinations and Bases	28
2.1.4	Rank	28
2.1.5	The Four Fundamental Subspaces	29
2.2	LU Factorization	29
2.2.1	LU and Gaussian Elimination	29
2.2.2	Pivoting and Stability	30
2.2.3	Algorithm Cost Analysis	30
2.3	Symmetric Positive Definite (SPD) Matrices	31
2.3.1	Definitions and Properties	31
2.3.2	Geometry and Metric	31
2.3.3	Cholesky Decomposition	32
3	Orthogonality and Least Squares	33
3.1	Orthogonal Matrices	33
3.1.1	Orthogonality and Orthonormal Bases	33
3.1.2	QR Decomposition	34
3.1.3	Gram-Schmidt Process	34
3.1.4	Orthogonal Projection	35
3.2	Vector and Matrix Norms	35
3.2.1	Vector Norms	35
3.2.2	Matrix Norms	36
3.3	Stability and Condition Number	37
3.3.1	Why Factorize instead of Invert?	37
3.3.2	Numerical Stability	37
3.3.3	Condition Number	37
3.3.4	Stability Hierarchy	38
3.4	Overdetermined Systems and Least Squares	38
3.4.1	Least Squares Problem	39
3.4.2	Normal Equations	39
3.4.3	QR-Based Least Squares	39
3.4.4	Polynomial Regression	40
3.5	Chebyshev Polynomials	40
3.5.1	Failure of the Monomial Basis	40
3.5.2	Chebyshev Polynomials (T_k)	40
3.5.3	Chebyshev Nodes	41
3.5.4	Chebyshev Series	41
4	Eigenvalues and the SVD	41
4.1	Eigenvalues and Eigenvectors	41
4.1.1	Definitions and Basic Properties	42
4.1.2	Symmetric and Hermitian Matrices	42

4.1.3	Iterative Algorithms	43
4.1.4	Schur Decomposition and the QR Algorithm	43
4.2	Singular Value Decomposition	44
4.2.1	The SVD Theorem	44
4.2.2	The Four Fundamental Subspaces	45
4.2.3	Low-Rank Approximation	45
4.2.4	The Pseudoinverse	45
4.3	Lanczos Algorithm	46
4.3.1	Krylov Subspaces	46
4.3.2	The Lanczos Recurrence	46
4.3.3	Ritz Values and Vectors	46
4.4	Jacobi Eigenvalue Algorithm	47
4.4.1	Givens Rotations	47
4.4.2	Convergence and Cost	47
5	Iterative Methods for Linear Systems	48
5.1	Iterative Methods for Linear Systems	48
5.1.1	Direct vs. Iterative Solvers	48
5.1.2	Convergence and Spectral Radius	49
5.1.3	Fill-in and Sparse Storage	49
5.2	Jacobi and Gauss-Seidel Methods	49
5.2.1	Jacobi Iteration	49
5.2.2	Gauss-Seidel Iteration	50
5.2.3	Convergence Conditions	50
5.2.4	Successive Over-Relaxation (SOR)	50
5.2.5	Stopping Criteria	51
5.3	Conjugate Gradient Method	51
5.3.1	From Linear Systems to Optimization	51
5.3.2	Conjugate Directions	52
5.3.3	The CG Algorithm	52
5.3.4	Convergence Rate	52
5.3.5	Preconditioning	53
5.4	GMRES	53
5.4.1	The Arnoldi Process	53
5.4.2	GMRES as Least Squares	54
5.4.3	Restarted GMRES(m)	54
5.4.4	Convergence and Preconditioning	54
6	Numerical Analysis	55
6.1	Numerical Quadrature	55
6.1.1	Newton-Cotes Rules	55
6.1.2	Gaussian Quadrature	55
6.1.3	Richardson Extrapolation and Romberg Integration	55
6.1.4	Adaptive Quadrature	56
6.1.5	Periodic Functions and Spectral Accuracy	56

6.1.6	Exercises	56
6.2	Root Finding	56
6.2.1	Bracketing Methods	56
6.2.2	Open Methods	57
6.2.3	Brent's Method	57
6.2.4	Exercises	58
7	Ordinary Differential Equations	58
7.1	Numerical Methods for ODEs	58
7.1.1	Basic Methods	58
7.1.2	Runge-Kutta Methods	58
7.1.3	Symplectic Integrators	59
7.1.4	Exercises	59
8	Applications	60
8.1	Principal Component Analysis (PCA)	60
8.1.1	Definitions and Preprocessing	60
8.1.2	PCA via SVD	60
8.1.3	Variance and Dimensionality Reduction	60
8.1.4	Implementation Details	61
8.1.5	Exercises	61
8.2	Spectral Graph Theory and PageRank	61
8.2.1	Graph Matrices	61
8.2.2	Spectral Clustering	62
8.2.3	Random Walks and PageRank	62
8.2.4	Exercises	63
8.3	Linear Operators and the Poisson Equation	63
8.3.1	Differential Operators and Discretization	63
8.3.2	Operator Norms and Conditioning	63
8.3.3	Exercises	64
8.4	Fourier Analysis as Change of Basis	64
8.4.1	Discrete Fourier Transform (DFT)	64
8.4.2	Convolution and Circulant Matrices	64
8.4.3	The Fast Fourier Transform (FFT)	65
8.4.4	Exercises	65
8.5	Markov Chains and Stochastic Matrices	65
8.5.1	Definitions	65
8.5.2	Convergence and Mixing	66
8.5.3	Properties and Design	66
8.5.4	Exercises	67
8.6	Koopman Theory, DMD, and SINDy	67
8.6.1	Koopman Operator Theory	67
8.6.2	Dynamic Mode Decomposition (DMD)	67
8.6.3	SINDy: Sparse Identification	68
8.6.4	Exercises	68

Preface

These notes started as a study guide for CS 111 at UC Santa Barbara and grew through my PhD as teaching and research kept exposing things I thought I understood until I tried to explain them: why LU factorization is numerically stable, why the QR algorithm converges, what the Koopman operator actually is. Writing forces the gaps to surface. The result is a reference written by a graduate student for himself. Treat it accordingly.

For a rigorous treatment of this material, see [Principles of Scientific Computing](#) by Bindel and Goodman, and G. W. Stewart's [Afternotes on Numerical Analysis](#). For further reading, the [CS 6241 reading list](#) at Cornell covers the literature well.

Computers were built to do numerical computation. The word “computer” originally meant a person whose job was to evaluate functions by hand. The U.S. Army employed hundreds of them during the Second World War to compute artillery firing tables, a task so large it directly motivated the construction of [ENIAC](#) (1945). The machine's first classified application was thermonuclear weapon simulation, directed by John von Neumann, whose [1947 paper with Goldstine](#) laid the foundations of numerical stability analysis. In 1950, Jule Charney used ENIAC to run the [first machine weather forecast](#): [Lewis Fry Richardson](#) had estimated the same computation would require 64,000 human computers working in parallel. The [Apollo guidance computer](#) navigated to the Moon in real time with 4 kilobytes of memory. The [Cooley-Tukey FFT](#) (1965) reduced a core signal processing computation from $O(n^2)$ to $O(n \log n)$. The demand for numerical computation at scale was the original motivation for programmable machines.

Almost every numerical algorithm reduces a nonlinear problem to a sequence of linear ones, because linear systems are the only class of problems we understand completely: their solution structure, their sensitivity to perturbations, their reliable algorithms. Newton's method, finite element discretization, PCA, gradient-based optimization, each is, at its core, a repeated linearization. Linear algebra is the substrate of scientific computing.

Much of the perspective here came from **CS 210A: Matrix Analysis**, taught by Professor Shivkumar Chandrasekaran in the ECE department at UC Santa Barbara. If you have the chance to take that course, take it.

Aditya Dendukuri
UC Santa Barbara, 2025

1 Fundamentals

1.1 Mathematical Notation and Proof Techniques

Standard vocabulary for proofs and algorithms.

1.1.1 Logic and Set Notation

Def 1.1 (Logical Quantifiers)

- **Universal ($\forall$):** “For all.”
- **Existential ($\exists$):** “There exists.”
- **Example:** “ $\forall \varepsilon > 0, \exists \delta > 0$ ” reads as “for every positive ε there is a positive δ .”

Def 1.2 (Implication and Equivalence)

- **Implication ($P \Rightarrow Q$):** P implies Q .
- **Equivalence ($P \Leftrightarrow Q$):** P if and only if Q (iff).
- **Contrapositive:** $\neg Q \Rightarrow \neg P$. Logically equivalent to $P \Rightarrow Q$.

Def 1.3 (Set Notation)

- $x \in S$: x is an element of S .
- $S \subseteq T$: S is a subset of T .
- $S \cup T$: Union; $S \cap T$: Intersection; $S \setminus T$: Set difference.
- $\{x \in S : P(x)\}$: Set builder notation.
- **Number Sets:** $\mathbb{Z}$ (integers), $\mathbb{R}$ (reals), $\mathbb{C}$ (complex), $\mathbb{R}^n$ (vectors), $\mathbb{R}^{m \times n}$ (matrices).

Ex 1.1

1. Write “every invertible matrix has a nonzero determinant” using quantifiers and implication.
2. State the contrapositive of: “if $\mathbf{A}$ is invertible, then $\mathbf{A}\mathbf{x} = \mathbf{0} \Rightarrow \mathbf{x} = \mathbf{0}$.”
3. Let $S = \{1, 2, 3, 4\}$ and $T = \{3, 4, 5, 6\}$. Compute $S \cup T, S \cap T, S \setminus T$.

1.1.2 Proof Techniques

Def 1.4 (Direct Proof) Assume P is true and derive Q through a chain of valid logical steps.

Def 1.5 (Proof by Contradiction) Assume $\neg P$ and derive a logical contradiction $(R \wedge \neg R)$. Proves P must hold.

Def 1.6 (Proof by Induction) To prove $P(n)$ holds for all integers $n \geq n_0$:

1. **Base case:** Verify $P(n_0)$ directly.
2. **Inductive step:** Assume $P(k)$ and prove $P(k+1)$.

Note 1.1 Numerical Proofs: Often validate that an approximation is **stable** or that an error term **converges**. Frequently involves showing $|f(x) - \hat{f}(x)| \leq \varepsilon$ for small ε .

Ex 1.2

1. Prove: if $\mathbf{A}$ and $\mathbf{B}$ are symmetric, then $\mathbf{A} + \mathbf{B}$ is symmetric.
2. Prove by contradiction: if $\mathbf{A}\mathbf{x} = \mathbf{0} \Rightarrow \mathbf{x} = \mathbf{0}$, then $\mathbf{A}$ has no zero eigenvalue.
3. By induction, prove $\sum_{k=1}^n k = \frac{n(n+1)}{2}$.

1.1.3 Notation Conventions

Note 1.2 Scalars, Vectors, and Matrices.

- **Scalars:** Lowercase italic (a, b, x).
- **Vectors:** Lowercase bold ($\mathbf{x}, \mathbf{y}, \mathbf{a}$). Default is **column vector**.
- **Matrices:** Uppercase bold ($\mathbf{A}, \mathbf{B}, \mathbf{Q}$).
- **Indexing:** 1-based (unlike Python). x_i is i -th entry; a_{ij} is (i, j) entry.

Note 1.3 Transpose and Inverse.

- $\mathbf{A}^T$: Transpose.
- $\mathbf{A}^{-1}$: Matrix inverse (square, nonsingular matrices).
- $\mathbf{A}^{-T}$: Transpose of the inverse $(\mathbf{A}^{-1})^T$.
- $\mathbf{A}^k$: k -th power. $\mathbf{A}^0 = \mathbf{I}$.

Note 1.4 Sums and Products.

$$\sum_{i=1}^n a_i = a_1 + \cdots + a_n, \quad \prod_{i=1}^n a_i = a_1 \cdots a_n.$$

Note 1.5 Functions.

- $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$: Maps n -vectors to m -vectors.
- $f \circ g$: Function composition $(f \circ g)(\mathbf{x}) = f(g(\mathbf{x}))$.

Note 1.6 **Asymptotic Notation.** Scaling of algorithms is described using ****Big-O notation**** ($O(n^2)$, $O(n^3)$). Details in the Computational Complexity section.

1.2 Vectors and Matrices

Scientific computing is built on two fundamental data structures: vectors and matrices. Before introducing these formally, we fix notation for the standard number systems that provide the raw material for all numerical computation.

Standard Number Systems. Throughout these notes, $\mathbb{R}$ denotes the set of all real numbers, $\mathbb{Z}$ the set of all integers, and $\mathbb{C}$ the set of all complex numbers. Our primary focus is real-valued computation; complex numbers arise naturally in eigenvalue analysis and signal processing.

Def 1.7 (**Scalar**) An element of a number system ($\mathbb{R}$, $\mathbb{Z}$, or $\mathbb{C}$) is called a **scalar**. Scalars represent individual numerical quantities and serve as the atomic building blocks from which vectors and matrices are constructed.

Ex 1.3 Refer to Def 1.7.

1. Classify each of the following as a real scalar, an integer, a complex (non-real) scalar, or *not* a scalar: 3.14; -7 ; $2 + 3i$; $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$.
2. In Python, evaluate `type(3.14)`, `type(3)`, and `type(3+2j)`. Which NumPy dtype would you use to store each?

Def 1.8 (**Vectors**) A **vector** is a one-dimensional ordered array of scalars. Vectors can represent points, directions, or more abstract quantities within a vector space, and are fundamental in encoding data and linear relationships.

Note 1.7 Vectors may be written as row vectors or column vectors. Although both forms carry the same information, the distinction matters when applying linear transformations and computing inner products. **In these notes, we assume vectors are column vectors by default.**

Example

$$(1 \ 2 \ 3), \quad \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad \begin{pmatrix} 1+2i \\ 7i \\ 3 \end{pmatrix}, \dots$$

Ex 1.4 Refer to Def 1.8.

1. Write a column vector $\mathbf{x} \in \mathbb{R}^4$ whose i -th entry equals 2^i for $i = 1, 2, 3, 4$.
2. In NumPy, construct this vector with `np.array(...)` and confirm `x.shape == (4,)`. **Why does NumPy report shape (4,) rather than (4,1)?** This is known as a *rank-1 array*; it is neither a row nor a column vector, and its broadcasting behavior differs from 2D arrays.
3. Can a vector in $\mathbb{R}^3$ be added to a vector in $\mathbb{R}^4$? Justify in one sentence.

Just as scalars belong to number systems like $\mathbb{R}$ or $\mathbb{C}$, vectors inhabit **vector spaces**. While these notes treat vector spaces informally, subsequent chapters provide more rigorous definitions and properties.

Def 1.9 ($\mathbb{R}^n$) The set of all real vectors of size n . When we write $\mathbf{x} \in \mathbb{R}^n$, it means $\mathbf{x}$ has n real entries. Vectors in $\mathbb{R}^n$ take one of the following forms:

$$\mathbf{x} \in \mathbb{R}^{n \times 1} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \quad (\text{column vector, default convention})$$
$$\mathbf{x} \in \mathbb{R}^{1 \times n} = (x_1 \ x_2 \ \cdots \ x_n) \quad (\text{row vector})$$

Note 1.8 These notes focus on finite-dimensional spaces. Infinite-dimensional vector spaces, central to functional analysis, arise in differential equations and quantum mechanics but are beyond our scope here.

Ex 1.5 Refer to Def 1.9.

1. Let $\mathbf{x} = (1, -2, 3)^T \in \mathbb{R}^3$. Identify the value of entry x_2 . What is the dimension of $\mathbb{R}^3$?
2. In NumPy, create a column vector (shape (3,1)) from the list [1, -2, 3] in two ways: using `np.reshape` and using `[:, np.newaxis]`.
3. What does `x.T` return when `x.shape == (3,)`? How does the result differ when `x.shape == (3,1)`?

Def 1.10 (Matrices) A **matrix** is a two-dimensional array of scalars. A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is written as

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}.$$

Matrices are used to represent linear transformations, systems of linear equations, and more complex data structures.

Example Two-dimensional matrices include:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \in \mathbb{R}^{2 \times 3}, \quad \begin{pmatrix} 1 & 2 \\ 4 & 5 \\ 6 & 7 \end{pmatrix} \in \mathbb{R}^{3 \times 2}.$$

Note 1.9 Structural Intuition: Matrix Columns. Matrices can be interpreted as vectors whose entries are themselves vectors:

$$\mathbf{A} = \begin{pmatrix} \text{---} \mathbf{r}_1 \text{---} \\ \vdots \\ \text{---} \mathbf{r}_n \text{---} \end{pmatrix} = \begin{pmatrix} | & & | \\ \mathbf{c}_1 & \cdots & \mathbf{c}_n \\ | & & | \end{pmatrix},$$

where $r_i = \mathbf{A}[i, :]$ is the i -th row and $c_i = \mathbf{A}[:, i]$ is the i -th column. This viewpoint is vital: the product $\mathbf{A}\mathbf{x}$ is simply a **linear combination of the columns of $\mathbf{A}$** weighted by the entries of $\mathbf{x}$.

Ex 1.6 Refer to Def 1.10.

1. State the dimensions (m, n) of $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$. What is the shape of its second column as a vector?
2. In NumPy (0-based indexing), write the expression to access entry a_{12} (first row, second column in 1-based notation). Then extract the second column of $\mathbf{A}$ and verify its shape is $(2,)$.
3. True or False: a vector in $\mathbb{R}^n$ is a special case of a matrix in $\mathbb{R}^{n \times 1}$. How does NumPy handle this distinction in practice?

Def 1.11 (Matrix Transpose) The **transpose** of $\mathbf{A} \in \mathbb{R}^{m \times n}$, denoted $\mathbf{A}^T$, is the $n \times m$ matrix obtained by interchanging rows and columns:

$$(\mathbf{A}^T)_{ij} = a_{ji}, \quad 1 \leq i \leq n, \quad 1 \leq j \leq m.$$

For a column vector $\mathbf{x} \in \mathbb{R}^n$, the transpose $\mathbf{x}^T$ is its corresponding row vector and vice versa.

Def 1.12 (Symmetric and Skew-Symmetric Matrices) A square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is:

- **symmetric** if $\mathbf{A}^T = \mathbf{A}$, equivalently $a_{ij} = a_{ji}$ for all i, j .
- **skew-symmetric** if $\mathbf{A}^T = -\mathbf{A}$, equivalently $a_{ij} = -a_{ji}$ for all i, j .

Note 1.10 Every square matrix $\mathbf{A}$ decomposes uniquely into a symmetric part and a skew-symmetric part:

$$\mathbf{A} = \underbrace{\frac{1}{2}(\mathbf{A} + \mathbf{A}^T)}_{\text{symmetric}} + \underbrace{\frac{1}{2}(\mathbf{A} - \mathbf{A}^T)}_{\text{skew-symmetric}}.$$

Symmetric matrices arise constantly in scientific computing: covariance matrices in statistics, graph Laplacians in network problems, and Hessians of quadratic objectives in optimization are all symmetric.

Thm 1.1 (Properties of the Transpose) Let $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times p}$, and $\alpha \in \mathbb{R}$. Then:

1. $(\mathbf{A}^T)^T = \mathbf{A}$
2. $(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T$ (for conformable $\mathbf{A}, \mathbf{B}$)
3. $(\alpha \mathbf{A})^T = \alpha \mathbf{A}^T$
4. $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$ (**Reversal Law**)

Proof *Proof.* Properties (1)–(3) follow directly from the definition $(\mathbf{A}^T)_{ij} = a_{ji}$ applied entry-wise. For example, for (2):

$$(\mathbf{A} + \mathbf{B})_{ij}^T = (\mathbf{A} + \mathbf{B})_{ji} = a_{ji} + b_{ji} = (\mathbf{A}^T)_{ij} + (\mathbf{B}^T)_{ij} = (\mathbf{A}^T + \mathbf{B}^T)_{ij}.$$

For (4), let $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times p}$. By definition of transpose and matrix product:

$$(\mathbf{C}^T)_{ij} = c_{ji} = \sum_{k=1}^n a_{jk} b_{ki}.$$

Expanding $\mathbf{B}^T \mathbf{A}^T$ using $(\mathbf{B}^T)_{ik} = b_{ki}$ and $(\mathbf{A}^T)_{kj} = a_{jk}$:

$$(\mathbf{B}^T \mathbf{A}^T)_{ij} = \sum_{k=1}^n (\mathbf{B}^T)_{ik} (\mathbf{A}^T)_{kj} = \sum_{k=1}^n b_{ki} a_{jk}.$$

Both sums are identical, so $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$. □

Cor 1.2 (Reversal Law for Multiple Factors) By induction on Thm 1.1(4):

$$(\mathbf{A}_1 \mathbf{A}_2 \cdots \mathbf{A}_k)^T = \mathbf{A}_k^T \cdots \mathbf{A}_2^T \mathbf{A}_1^T.$$

Ex 1.7 Refer to Def 1.11 and Thm 1.1. Let $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$.

1. State the dimensions of $\mathbf{A}$, $\mathbf{A}^T$, $\mathbf{A}\mathbf{A}^T$, and $\mathbf{A}^T\mathbf{A}$.
2. Compute $\mathbf{A}^T$ explicitly. Then compute $\mathbf{A}\mathbf{A}^T$ and verify it is symmetric using Def 1.12.
3. In NumPy (0-based indexing), write the expression to retrieve element a_{12} (first row, second column in 1-based notation). Then write a one-liner for $\mathbf{A}\mathbf{A}^T$.
4. Is $\mathbf{A}\mathbf{A}^T = \mathbf{A}^T\mathbf{A}$ in general? What can you say about each product individually?

Cor 1.3 (Outer Product is Symmetric of Rank 1) For any nonzero $\mathbf{x} \in \mathbb{R}^n$, the outer product $\mathbf{x}\mathbf{x}^T \in \mathbb{R}^{n \times n}$ is symmetric and has rank 1. The inner product $\mathbf{x}^T\mathbf{x} \in \mathbb{R}$ satisfies $\mathbf{x}^T\mathbf{x} \geq 0$ with equality iff $\mathbf{x} = \mathbf{0}$.

Ex 1.8 In this exercise, we will prove Cor 1.3. Refer to Def 1.11 and Thm 1.1. Let $\mathbf{x} = (x_1, \dots, x_n)^T \in \mathbb{R}^n$ be a nonzero column vector.

1. The **outer product** is $\mathbf{x}\mathbf{x}^T \in \mathbb{R}^{n \times n}$. Write out entry (i, j) explicitly.
2. Use the Reversal Law (Thm 1.1(4)) to show $\mathbf{x}\mathbf{x}^T$ is symmetric.
3. Show that $\mathbf{x}\mathbf{x}^T$ has rank 1 by arguing about the column space. (*Hint: every column of $\mathbf{x}\mathbf{x}^T$ is a scalar multiple of $\mathbf{x}$.*)
4. Express $\mathbf{x}^T\mathbf{x}$ as a sum of squares. Conclude $\mathbf{x}^T\mathbf{x} \geq 0$ with equality iff $\mathbf{x} = \mathbf{0}$, and identify the geometric quantity $\sqrt{\mathbf{x}^T\mathbf{x}}$.

1.3 Matrix Arithmetic

We are all familiar with scalar arithmetic: addition, subtraction, multiplication, and properties such as associativity and distributivity. In the following sections we extend these operations to vectors and matrices, building the algebraic machinery used throughout scientific computing.

1.3.1 Addition and Subtraction

Matrix addition generalizes scalar addition by operating entry-wise. The key constraint is dimensional compatibility: two matrices can be added only when they share the same shape.

Def 1.13 (Matrix Sum) Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$. Their sum is defined as

$$\mathbf{A} + \mathbf{B} = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{pmatrix} + \begin{pmatrix} b_{11} & \cdots & b_{1n} \\ \vdots & \ddots & \vdots \\ b_{n1} & \cdots & b_{nn} \end{pmatrix} = \begin{pmatrix} (a+b)_{11} & \cdots & (a+b)_{1n} \\ \vdots & \ddots & \vdots \\ (a+b)_{n1} & \cdots & (a+b)_{nn} \end{pmatrix},$$

where each entry satisfies $(a+b)_{ij} = a_{ij} + b_{ij}$ for $1 \leq i \leq m$ and $1 \leq j \leq n$.

Note 1.11 For $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{p \times q}$, the sum $\mathbf{A} + \mathbf{B}$ is defined if and only if $m = p$ and $n = q$.

Ex 1.9 Refer to Def 1.13.

1. Let $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$. Compute $\mathbf{A} + \mathbf{B}$ and $2\mathbf{A} - \mathbf{B}$ by hand.
2. In NumPy, verify your answers using `np.array`. Then check whether `np.array([1,2,3]) + np.array([1,2])` raises an error and explain why.

Commutativity and Associativity of Matrix Addition

Let

$$\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \quad \text{and} \quad \mathbf{B} = \begin{pmatrix} e & f \\ g & h \end{pmatrix}.$$

Then,

$$\mathbf{A} + \mathbf{B} = \begin{pmatrix} a+e & b+f \\ c+g & d+h \end{pmatrix} \quad \text{and} \quad \mathbf{B} + \mathbf{A} = \begin{pmatrix} e+a & f+b \\ g+c & h+d \end{pmatrix}.$$

Since addition of real numbers is commutative, $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$. Furthermore, for any third matrix $\mathbf{C} = \begin{pmatrix} i & j \\ k & l \end{pmatrix}$,

$$(\mathbf{A} + \mathbf{B}) + \mathbf{C} = \begin{pmatrix} (a+e)+i & (b+f)+j \\ (c+g)+k & (d+h)+l \end{pmatrix},$$

and

$$\mathbf{A} + (\mathbf{B} + \mathbf{C}) = \begin{pmatrix} a+(e+i) & b+(f+j) \\ c+(g+k) & d+(h+l) \end{pmatrix}.$$

By associativity of real addition, these are identical. Hence matrix addition is both commutative and associative.

Thm 1.4 ($\mathbb{R}^{m \times n}$ as a Vector Space) The set $\mathbb{R}^{m \times n}$ equipped with matrix addition and scalar multiplication forms a real vector space of dimension mn .

Proof *Proof.* Each axiom follows from the corresponding property of $\mathbb{R}$ applied entry-wise. For **commutativity**:

$$(\mathbf{A} + \mathbf{B})_{ij} = a_{ij} + b_{ij} = b_{ij} + a_{ij} = (\mathbf{B} + \mathbf{A})_{ij}.$$

The **additive identity** is $\mathbf{0}$ since $(\mathbf{A} + \mathbf{0})_{ij} = a_{ij}$. The **additive inverse** of $\mathbf{A}$ is $-\mathbf{A}$ (negate every entry), satisfying $(\mathbf{A} + (-\mathbf{A}))_{ij} = 0$.

For the **dimension**: the mn matrices $\mathbf{E}^{(ij)}$ with a 1 in position (i, j) and zeros elsewhere are linearly independent, and every $\mathbf{A} \in \mathbb{R}^{m \times n}$ decomposes as $\mathbf{A} = \sum_{i,j} a_{ij} \mathbf{E}^{(ij)}$, so they form a basis of size mn . $\square$

Def 1.14 (Scalar Multiplication) Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $c \in \mathbb{R}$. Then scalar multiplication is defined as

$$c\mathbf{A} = c \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{pmatrix} = \begin{pmatrix} ca_{11} & \cdots & ca_{1n} \\ \vdots & \ddots & \vdots \\ ca_{n1} & \cdots & ca_{nn} \end{pmatrix},$$

where each entry satisfies $(ca)_{ij} = c \cdot a_{ij}$.

Distributivity of Scalar Multiplication over Matrix Addition

Let $\alpha \in \mathbb{R}$ and

$$\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} e & f \\ g & h \end{pmatrix}.$$

Then,

$$\alpha(\mathbf{A} + \mathbf{B}) = \alpha \begin{pmatrix} a+e & b+f \\ c+g & d+h \end{pmatrix} = \begin{pmatrix} \alpha(a+e) & \alpha(b+f) \\ \alpha(c+g) & \alpha(d+h) \end{pmatrix}.$$

Since scalar multiplication distributes over addition for real numbers, this equals

$$\begin{pmatrix} \alpha a + \alpha e & \alpha b + \alpha f \\ \alpha c + \alpha g & \alpha d + \alpha h \end{pmatrix} = \alpha \mathbf{A} + \alpha \mathbf{B}.$$

Ex 1.10 Refer to Def 1.14 and Thm 1.4.

1. Compute $3\mathbf{A} - 2\mathbf{B}$ for $\mathbf{A} = \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 2 & 1 \\ 0 & -1 \end{pmatrix}$.
2. The standard basis for $\mathbb{R}^{2 \times 2}$ consists of four matrices $\mathbf{E}^{(ij)}$. Write them out explicitly and verify that $\mathbf{A} = \sum_{i,j} a_{ij} \mathbf{E}^{(ij)}$ holds for your $\mathbf{A}$ above.

1.3.2 Dot Product

Before defining the general matrix product, we introduce the dot product of two vectors. The matrix product will then be understood entry-wise as a collection of dot products.

Def 1.15 (Dot Product) For vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$, the **dot product** (or inner product) is defined as

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b} = (a_1 \quad \cdots \quad a_n) \begin{pmatrix} b_1 \\ \vdots \\ b_n \end{pmatrix} = \sum_{i=1}^n a_i b_i. \quad (1)$$

Ex 1.11 Refer to Def 1.15.

1. Compute $\mathbf{a} \cdot \mathbf{b}$ for $\mathbf{a} = (1, 2, 3)^T$ and $\mathbf{b} = (-1, 0, 2)^T$ by hand.
2. In NumPy, evaluate this using `np.dot(a, b)` and also with `a @ b`. Confirm both give the same result.

3. Show that $\mathbf{a} \cdot \mathbf{a} \geq 0$ with equality iff $\mathbf{a} = \mathbf{0}$. What geometric quantity does $\sqrt{\mathbf{a} \cdot \mathbf{a}}$ represent?

Def 1.16 (Matrix Product) Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$. Their product $\mathbf{AB}$ is an $m \times p$ matrix defined by

$$(\mathbf{AB})_{ij} = \sum_{k=1}^n a_{ik}b_{kj},$$

or equivalently, if $\mathbf{a}_i$ denotes the i -th row of $\mathbf{A}$ and $\mathbf{b}_j$ the j -th column of $\mathbf{B}$:

$$\mathbf{AB} = \begin{pmatrix} \mathbf{a}_1 \cdot \mathbf{b}_1 & \cdots & \mathbf{a}_1 \cdot \mathbf{b}_p \\ \vdots & \ddots & \vdots \\ \mathbf{a}_m \cdot \mathbf{b}_1 & \cdots & \mathbf{a}_m \cdot \mathbf{b}_p \end{pmatrix}.$$

Note 1.12 Matrix multiplication is defined only when the number of columns of $\mathbf{A}$ equals the number of rows of $\mathbf{B}$. It is associative and distributive over addition, but in general not commutative: $\mathbf{AB} \neq \mathbf{BA}$ for most matrices.

Lem 1.5 (Computational Cost of Matrix Multiplication) Computing $\mathbf{AB}$ where $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$ requires exactly mnp multiplications and $m(n-1)p$ additions using the standard algorithm, giving $O(mnp)$ floating-point operations (flops). For two square $n \times n$ matrices this reduces to $O(n^3)$ flops.

Note 1.13 The $O(n^3)$ cost of naive matrix multiplication is a central bottleneck in scientific computing. Strassen's algorithm reduces this to $O(n^{\log_2 7}) \approx O(n^{2.807})$, but is rarely used in practice due to numerical stability concerns and cache-unfriendly memory access. Modern BLAS/LAPACK implementations approach peak hardware throughput via blocking strategies, but the asymptotic cost remains $O(n^3)$. For many applications, avoiding explicit matrix-matrix products entirely (e.g., exploiting sparsity or factored structure) is more impactful than algorithmic improvements.

Compatibility for Matrix Multiplication

Let $\mathbf{A}$ be a 3×4 matrix and $\mathbf{B}$ a 4×2 matrix. Since the number of columns of $\mathbf{A}$ (4) equals the number of rows of $\mathbf{B}$ (4), the product $\mathbf{AB}$ is defined and results in a 3×2 matrix. Conversely, if $\mathbf{B}$ were a 5×2 matrix, the product would be undefined.

Distributive Property of Matrix Multiplication

Let

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}.$$

Then,

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{A} \begin{pmatrix} b_{11} + c_{11} & b_{12} + c_{12} \\ b_{21} + c_{21} & b_{22} + c_{22} \end{pmatrix},$$

and by computing the entries, one can verify that

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}.$$

Non-Commutativity of Matrix Multiplication

Consider

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} \quad \text{and} \quad \mathbf{B} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Then,

$$\mathbf{AB} = \begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix} \quad \text{and} \quad \mathbf{BA} = \begin{pmatrix} 0 & 2 \\ 1 & 0 \end{pmatrix}.$$

Since $\mathbf{AB} \neq \mathbf{BA}$, matrix multiplication is not commutative.

Ex 1.12 Refer to Def 1.16 and Lem 1.5. Let $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 1 & -1 \\ 2 & 4 \end{pmatrix}$.

1. Compute $\mathbf{AB}$ and $\mathbf{BA}$ by hand. Confirm $\mathbf{AB} \neq \mathbf{BA}$.
2. Find a nonzero matrix $\mathbf{C} \neq \mathbf{I}$ that commutes with $\mathbf{A}$ (i.e., $\mathbf{AC} = \mathbf{CA}$). *Hint: try $\mathbf{C} = \alpha\mathbf{A} + \beta\mathbf{I}$.*
3. Verify associativity: compute $(\mathbf{AB})\mathbf{c}$ and $\mathbf{A}(\mathbf{Bc})$ for $\mathbf{c} = (1, 0)^T$ and confirm they are equal.
4. Compare the flop count of $(\mathbf{AB})\mathbf{c}$ versus $\mathbf{A}(\mathbf{Bc})$ for general $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ and $\mathbf{c} \in \mathbb{R}^n$. Which order is cheaper and by what factor?

Many properties of scalar arithmetic extend naturally to vectors and matrices. In fact, matrices can be viewed as elements of a higher-dimensional vector space, and matrix multiplication can be interpreted as the composition of linear transformations. While the notations for vectors and matrices are similar, their roles differ:

- Vectors typically represent points or directions in space.
- Matrices often represent linear maps or transformations between vector spaces.

This conceptual distinction becomes crucial when discussing eigenvalues and linear transformations.

Thm 1.6 (Associativity as an Optimization) For $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ and $\mathbf{x} \in \mathbb{R}^n$, computing $\mathbf{A}(\mathbf{Bx})$ via two matrix-vector products costs $O(n^2)$ flops, whereas forming $(\mathbf{AB})\mathbf{x}$ via a matrix-matrix product costs $O(n^3)$ flops.

Ex 1.13 Suppose $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ and $\mathbf{x} \in \mathbb{R}^n$.

1. *Approach 1:* Compute $\mathbf{M} = \mathbf{AB}$ first (cost?), then $\mathbf{Mx}$ (cost?). Write the total flop count.

2. *Approach 2:* Compute $\mathbf{y} = \mathbf{B}\mathbf{x}$ first (cost?), then $\mathbf{A}\mathbf{y}$ (cost?). Write the total flop count.
3. Conclude the asymptotic improvement factor as $n \rightarrow \infty$.
4. Write NumPy code for both approaches using `@`. Does NumPy automatically choose the optimal evaluation order for a chain like $\mathbf{A} @ \mathbf{B} @ \mathbf{x}$?

1.3.3 Matrix Inverse

We now turn to a fundamental question: when can we “divide” by a matrix? The matrix inverse plays the role of reciprocal in scalar arithmetic, but it exists only under a specific condition, and computing it explicitly is almost always the wrong approach in practice.

Def 1.17 (Matrix Inverse) For a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, the **inverse** of $\mathbf{A}$, denoted $\mathbf{A}^{-1}$, is the unique matrix satisfying

$$\mathbf{A}\mathbf{A}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I},$$

where $\mathbf{I}$ is the $n \times n$ identity matrix. A matrix that has an inverse is called *invertible* or *nonsingular*.

Note 1.14 A square matrix $\mathbf{A}$ is invertible if and only if $\det(\mathbf{A}) \neq 0$, which also implies that $\mathbf{A}$ has full rank.

Uniqueness of the Inverse

Suppose $\mathbf{A}$ is invertible and both $\mathbf{B}$ and $\mathbf{C}$ satisfy the conditions of Def 1.17. Then,

$$\mathbf{B} = \mathbf{B}\mathbf{I} = \mathbf{B}(\mathbf{A}\mathbf{C}) = (\mathbf{B}\mathbf{A})\mathbf{C} = \mathbf{I}\mathbf{C} = \mathbf{C}.$$

Thus the inverse of $\mathbf{A}$ is unique.

Thm 1.7 (Properties of the Matrix Inverse) Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ be invertible and $\alpha \in \mathbb{R} \setminus \{0\}$. Then:

1. $(\mathbf{A}^{-1})^{-1} = \mathbf{A}$
2. $(\mathbf{A}\mathbf{B})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ (**Reversal Law**)
3. $(\alpha\mathbf{A})^{-1} = \frac{1}{\alpha}\mathbf{A}^{-1}$
4. $(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T$

Proof *Proof.* **(1):** $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$ and $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$ show $\mathbf{A}$ is both a left and right inverse of $\mathbf{A}^{-1}$, so $(\mathbf{A}^{-1})^{-1} = \mathbf{A}$.
(2): Check the right-inverse condition:

$$(\mathbf{A}\mathbf{B})(\mathbf{B}^{-1}\mathbf{A}^{-1}) = \mathbf{A}(\mathbf{B}\mathbf{B}^{-1})\mathbf{A}^{-1} = \mathbf{A}\mathbf{I}\mathbf{A}^{-1} = \mathbf{A}\mathbf{A}^{-1} = \mathbf{I}.$$

Left-inverse: $(\mathbf{B}^{-1}\mathbf{A}^{-1})(\mathbf{AB}) = \mathbf{B}^{-1}(\mathbf{A}^{-1}\mathbf{A})\mathbf{B} = \mathbf{I}$. By uniqueness of the inverse, $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$.

(3): $(\alpha\mathbf{A})(\frac{1}{\alpha}\mathbf{A}^{-1}) = \frac{\alpha}{\alpha}\mathbf{AA}^{-1} = \mathbf{I}$.

(4): Transpose both sides of $\mathbf{AA}^{-1} = \mathbf{I}$ using the Reversal Law for transposes: $(\mathbf{A}^{-1})^T\mathbf{A}^T = \mathbf{I}$, so $(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T$. $\square$

Note 1.15 The Decomposition Principle. In scientific computing, we almost never compute $\mathbf{A}^{-1}$ explicitly. To solve $\mathbf{Ax} = \mathbf{b}$, we decompose $\mathbf{A}$ into simpler factors (like LU or QR) and solve a sequence of simpler systems. This is more numerically stable and computationally efficient. The Reversal Law $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ reflects this: to solve $(\mathbf{AB})\mathbf{x} = \mathbf{b}$, we first solve $\mathbf{Ay} = \mathbf{b}$ for $\mathbf{y}$, then $\mathbf{Bx} = \mathbf{y}$.

Ex 1.14 Refer to Def 1.17 and Thm 1.7.

1. For $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 5 & 3 \end{pmatrix}$, compute $\mathbf{A}^{-1}$ by hand using the 2×2 formula $\mathbf{A}^{-1} = \frac{1}{\det \mathbf{A}} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$. Verify $\mathbf{AA}^{-1} = \mathbf{I}$.
2. Given $\mathbf{B} = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix}$, use the Reversal Law to compute $(\mathbf{AB})^{-1}$ without inverting a new 2×2 system.
3. Explain why `np.linalg.solve(A, b)` is preferred over `np.linalg.inv(A) @ b` for solving $\mathbf{Ax} = \mathbf{b}$. Address both numerical accuracy and computational cost.

Def 1.18 (Involutory Matrix) A square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is called **involutory** if $\mathbf{A}^2 = \mathbf{I}$.

Ex 1.15 In this exercise, we will prove that a matrix is involutory (Def 1.18) if and only if it is its own inverse. Refer to Def 1.17 and Thm 1.7.

1. Suppose $\mathbf{A}^2 = \mathbf{I}$. Multiply both sides of $\mathbf{AA} = \mathbf{I}$ on the right by $\mathbf{A}^{-1}$ to deduce $\mathbf{A}^{-1} = \mathbf{A}$.
2. Conversely, suppose $\mathbf{A}^{-1} = \mathbf{A}$. Show $\mathbf{A}^2 = \mathbf{I}$.
3. Give a concrete 2×2 example of an involutory matrix other than $\pm\mathbf{I}$. Verify it satisfies $\mathbf{A}^2 = \mathbf{I}$.

Ex 1.16 (Identifying Invertibility) For each matrix below, determine without computing the inverse whether it is invertible. Give a one-sentence justification.

$$(a) \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix} \quad (b) \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (c) \begin{pmatrix} 3 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & -1 \end{pmatrix} \quad (d) \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 2 & 1 \end{pmatrix}$$

1.4 Introduction to Norms

A *norm* is a function that assigns a non-negative scalar “size” to a vector or matrix. Norms are the fundamental tool for measuring errors, bounding perturbations, and defining convergence throughout scientific computing. This chapter introduces the key vector norm definitions and the families most used in practice. We will study norms more rigorously in later chapters, where we develop matrix norms, operator inequalities, Cauchy-Schwarz, Hölder’s inequality, and the condition number.

Def 1.19 (Vector Norm) A function $\|\cdot\| : \mathbb{R}^n \rightarrow \mathbb{R}$ is a *vector norm* if for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and $c \in \mathbb{R}$:

1. $\|\mathbf{x}\| \geq 0$ and $\|\mathbf{x}\| = 0 \Leftrightarrow \mathbf{x} = \mathbf{0}$ (positive definiteness)
2. $\|c\mathbf{x}\| = |c|\|\mathbf{x}\|$ (absolute homogeneity)
3. $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ (triangle inequality)

Def 1.20 (ℓ^p Norms) For $\mathbf{x} \in \mathbb{R}^n$ and $p \geq 1$, the ℓ^p norm is

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}.$$

The three most common cases are:

- $\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$ (*Manhattan norm* — sum of absolute values)
- $\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\mathbf{x}^T \mathbf{x}}$ (*Euclidean norm*)
- $\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|$ (*max norm*, the limit as $p \rightarrow \infty$)

Note 1.16 Physical Intuition. In scientific computing, the choice of norm depends on what you want to measure:

- ℓ_2 (**Energy**): The standard measure of magnitude; corresponds to physical energy or distance.
- ℓ_∞ (**Worst-case**): Measures the single largest entry. Useful for “tolerance” checks (e.g., “every entry must be below 10^{-6} ”).
- ℓ_1 (**Sparsity**): Encourages or measures sparsity; often used in optimization and compressed sensing.

Example For $\mathbf{x} = (3, -4, 0)^T$:

$$\|\mathbf{x}\|_1 = 3 + 4 + 0 = 7, \quad \|\mathbf{x}\|_2 = \sqrt{9 + 16 + 0} = 5, \quad \|\mathbf{x}\|_\infty = 4.$$

The unit balls $\{\mathbf{x} : \|\mathbf{x}\|_p \leq 1\}$ in $\mathbb{R}^2$ are: a diamond ($p = 1$), a circle ($p = 2$), and a square ($p = \infty$). As p increases, the unit ball grows toward the ℓ^∞ square.

Note 1.17 Throughout these notes, $\|\cdot\|$ without a subscript denotes the Euclidean norm $\|\cdot\|_2$ unless stated otherwise. Errors and residuals are almost always measured in $\|\cdot\|_2$. In NumPy: `np.linalg.norm(x)` computes $\|\mathbf{x}\|_2$; `np.linalg.norm(x, ord=p)` computes $\|\mathbf{x}\|_p$.

Note 1.18 **Numerical Zero.** In floating-point arithmetic, the condition $\|\mathbf{x}\| = 0$ is rarely met exactly due to rounding. We instead use a **tolerance** ε : we say $\mathbf{x}$ is “numerically zero” if $\|\mathbf{x}\| \leq \varepsilon \cdot \|\mathbf{x}_0\|$, where $\mathbf{x}_0$ is some initial reference vector.

Thm 1.8 **(All Norms are Equivalent on $\mathbb{R}^n$)** For any two norms $\|\cdot\|_\alpha$ and $\|\cdot\|_\beta$ on $\mathbb{R}^n$, there exist constants $c_1, c_2 > 0$ such that

$$c_1 \|\mathbf{x}\|_\beta \leq \|\mathbf{x}\|_\alpha \leq c_2 \|\mathbf{x}\|_\beta \quad \text{for all } \mathbf{x} \in \mathbb{R}^n.$$

In particular: $\|\mathbf{x}\|_2 \leq \|\mathbf{x}\|_1 \leq \sqrt{n} \|\mathbf{x}\|_2$ and $\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \sqrt{n} \|\mathbf{x}\|_\infty$.

Proof *Proof.* Since $\mathbb{R}^n$ is finite-dimensional, the unit sphere $\{\mathbf{x} : \|\mathbf{x}\|_\beta = 1\}$ is compact. Any norm is a continuous function, so it attains its minimum and maximum on a compact set — these extremal values are c_1 and $1/c_2$. The explicit inequalities follow from Cauchy-Schwarz: $\|\mathbf{x}\|_1 = \sum |x_i| \cdot 1 \leq \|\mathbf{x}\|_2 \|\mathbf{1}\|_2 = \sqrt{n} \|\mathbf{x}\|_2$. $\square$

Ex 1.17 Refer to Defs 1.19–1.20 and Thm 1.8.

1. For $\mathbf{x} = (-1, 2, -3, 4)^T$, compute $\|\mathbf{x}\|_1$, $\|\mathbf{x}\|_2$, $\|\mathbf{x}\|_\infty$ by hand. Verify all three norm-equivalence inequalities hold.
2. Verify the triangle inequality $\|\mathbf{x} + \mathbf{y}\|_2 \leq \|\mathbf{x}\|_2 + \|\mathbf{y}\|_2$ for $\mathbf{x} = (1, 2)^T$, $\mathbf{y} = (-3, 1)^T$ by computing both sides.
3. Using NumPy, confirm your hand calculations with `np.linalg.norm(x, 1)`, `np.linalg.norm(x)` and `np.linalg.norm(x, np.inf)`.
4. Show that $\|\mathbf{x}\|_\infty = \lim_{p \rightarrow \infty} \|\mathbf{x}\|_p$. (*Hint: let $M = \max_i |x_i|$; factor M out of $\|\mathbf{x}\|_p$ and take the limit.*)

1.5 Computational Complexity

Algorithmic cost measured in arithmetic operations (flops). Determines scalability and feasibility.

1.5.1 Big-O Notation

Def 1.21 (**Big-O Notation**) $f(n) = O(g(n))$ if $\exists C, n_0 > 0$ s.t. $|f(n)| \leq C|g(n)|$ for $n \geq n_0$.

- $\Omega(g)$: Grows at least as fast.
- $\Theta(g)$: Grows at the same rate.
- $o(g)$: Grows strictly slower.

Note 1.19 **Typical Rates.** $O(1) \ll O(\log n) \ll O(n) \ll O(n \log n) \ll O(n^2) \ll O(n^3) \ll O(2^n)$.
For $n = 1000$: $O(n^2)$ is $\sim 10^6$ ops (ms); $O(n^3)$ is $\sim 10^9$ ops (s).

Note 1.20 **Properties.** 1. **Transitivity:** $f = O(g), g = O(h) \Rightarrow f = O(h)$. 2. **Sum Rule:** $O(f) + O(g) = O(\max(f, g))$. 3. **Product Rule:** $O(f) \cdot O(g) = O(fg)$. 4. **Constant Factors:** $O(cf) = O(f)$.

Ex 1.18

1. Show $3n^2 + 100n + 5 = O(n^2)$.
2. Compare growth: $n \log n$ vs n^2 .
3. Rank by cost: $5n^3 + n, 100n^2, n^2 \log n, 2^{10}n^2$.
4. Use Master Theorem: $T(n) = 2T(n/2) + n$.

1.5.2 Flop Counting

Hardware-independent measure of work.

Def 1.22 (**Flop**) A single floating-point operation $(+, -, \times, \div)$.

Thm 1.9 (**Basic Costs**) For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{B} \in \mathbb{R}^{n \times p}$:

- **Dot product:** $2n$ flops.
- **Matrix-vector product:** $2mn$ flops.
- **Matrix-matrix product:** $2mnp$ flops ($2n^3$ for square).
- **Triangular solve:** n^2 flops.

Note 1.21 **Note: The Memory Wall.** In modern hardware, arithmetic is cheap but moving data (RAM to CPU) is expensive. An $O(n^3)$ algorithm with high cache locality (like blocked GEMM) can outperform an $O(n^2)$ algorithm with poor locality.

Note 1.22 **Benchmarks.**

- **LU Factorization:** $\frac{2}{3}n^3$.
- **Cholesky:** $\frac{1}{3}n^3$.
- **QR (Householder):** $\frac{4}{3}n^3$.

Ex 1.19

1. Verify $2mn$ flops for $\mathbf{A}\mathbf{x}$ by counting multiply-adds.
2. Estimate wall-clock time for 10^9 flops on a 10^9 flop/s CPU.
3. Python: Plot `np.dot(A, B)` time vs n (log-log). Verify exponent ≈ 3 .
4. Why might the empirical exponent be less than 3? (Cache hierarchy, vectorization).

1.6 Floating Point Arithmetic

Representation and precision limits of real numbers on digital hardware.

1.6.1 Floating Point Representation

Normalized numbers take the form $x = (-1)^s \times (1.f) \times 2^e$.

- **s:** Sign bit.
- **f:** Fractional significand (mantissa); leading 1 is implicit.
- **e:** Biased exponent; Bias = $2^{k-1} - 1$ (k is exponent bits).

Note 1.23 IEEE 754 Standards.

- **Single (32-bit):** 1 sign, 8 exponent, 23 fraction bits.
- **Double (64-bit):** 1 sign, 11 exponent, 52 fraction bits.

Example 16-bit float. 1 sign, 5 exp (bias 15), 10 frac. Represent -3.25 :

1. $s = 1$.
2. $3.25_{10} = 11.01_2 = 1.101_2 \times 2^1$.
3. $e = 1 + 15 = 16 = 10000_2$.
4. $f = 1010000000_2$.

Result: 1 10000 1010000000.

Ex 1.20

1. Encode +5.5 in 16-bit format.
2. How many distinct normalized numbers for 5 exp bits, 10 frac bits?

Def 1.23 (Machine Epsilon ($\varepsilon_{\text{mach}}$)) The difference between 1 and the next larger float. $\varepsilon_{\text{mach}} = 2^{1-p}$ where p is significand bits.

- Double precision: $\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$.

Ex 1.21

1. Python: Compare `1.0 + 1e-16` and `1.0 + 2e-16`.
2. Find $\varepsilon_{\text{mach}}$ empirically via loop.

Def 1.24 (Unit Roundoff (u)) Max relative rounding error: $u = \varepsilon_{\text{mach}}/2 = 2^{-p}$.

$$\frac{|\text{fl}(x) - x|}{|x|} \leq u.$$

Thm 1.10 (Fundamental Axiom) IEEE 754 guarantees $\text{fl}(a \circ b) = (a \circ b)(1 + \delta)$ for $\circ \in \{+, -, \times, \div\}$ and $|\delta| \leq u$. Individual operations are as accurate as the representation allows.

Note 1.24 Warning: After n operations, error grows as $O(nu)$. Operation order matters because each intermediate result is rounded.

Ex 1.22

1. Let $a = 1, b = -0.999 \dots$. Is error in $\text{fl}(a + b)$ bounded by u ?
2. Estimate worst-case relative error for 10^6 sequential additions.

1.6.2 IEEE 754 Special Values

- **Signed Zero:** $+0, -0$. $1/(+0) = \infty$, $1/(-0) = -\infty$.
- **Subnormals:** Fill the gap between 0 and the smallest normalized number. Reduced precision, allows gradual underflow.
- **Infinities:** $\pm\infty$. Result of overflow or $x/0$.
- **NaN:** Not a Number ($0/0, \infty - \infty$). Unequal to everything, including itself.

Ex 1.23

1. Python: `1.0/0.0` vs `1.0/-0.0`.
2. Result of `float('inf') - float('inf')` and `float('nan') == float('nan')`.

1.6.3 Arithmetic Pitfalls

Finite precision breaks standard algebraic laws:

- **Non-Associativity:** $(a + b) + c \neq a + (b + c)$.
- **Non-Distributivity:** $a(b + c) \neq ab + ac$.
- **Absorption:** $a + b = a$ when $|a| \gg |b|$.

Ex 1.24

1. `a=1e16, b=1`. Find `(a+b)-a` in Python. Explain result.
2. Find a, b, c where `a*(b+c) != a*b + a*c`.

1.7 Quantifying Approximation Error

1.7.1 Absolute and Relative Errors

- **Absolute:** $|\hat{x} - x|$.
- **Relative:** $\varepsilon = \frac{|\hat{x} - x|}{|x|}$. Standard measure for precision.

Ex 1.25

1. $\sqrt{2} \approx 1.414$. Errors?
2. Relative error of product $\hat{x}\hat{y}$? (Sum of individual relative errors).

Def 1.25 (Catastrophic Cancellation) Subtraction of nearly equal numbers leads to total loss of significant digits.

- Relative error $\leq u \cdot \frac{x+y}{x-y}$.
- Amplification factor $\rightarrow \infty$ as $x \rightarrow y$.

Note 1.25 Rule: Rounding error is inevitable. Cancellation is algorithmic and can often be eliminated via mathematical reformulation.

Example Stabilizing Cancellation. $f(x) = \sqrt{x+1} - \sqrt{x}$ for $x = 10^8$.

- **Direct:** $10000.00000005 - 10000 \approx 0$ (Catastrophic).
- **Stable:** $\frac{1}{\sqrt{x+1} + \sqrt{x}} \approx 5 \times 10^{-5}$.

Ex 1.26 Stabilize:

1. $(1 - \cos x)/x^2$ for small x .
2. Smaller root of $x^2 - 10^8x + 1 = 0$ via quadratic formula.
3. $e^x - 1$ for small x (Why use `np.expm1`?).

1.7.2 Error Analysis Frameworks

- **Forward:** Tracks error from inputs to outputs. Pessimistic.
- **Backward:** Asks: “What input perturbation would make the computed result exact?”
- **Backward Stability:** Algorithm solves a nearby problem exactly (Backward Error is $O(u)$).

Def 1.26 (**Condition Number** (κ)) Inherent sensitivity of the problem. Forward Error $\leq \kappa \cdot$ Backward Error.

Ex 1.27

1. $\kappa(A) = 10^8$, Backward Error = 10^{-15} . Max forward error?
2. Prove BE for linear systems: $(\mathbf{A} + \mathbf{E})\hat{\mathbf{x}} = \mathbf{b}$ for $\mathbf{E} = -\mathbf{r}\hat{\mathbf{x}}^T / \|\hat{\mathbf{x}}\|^2$.

1.8 Python and NumPy Primer

Condensed reference for the toolset used throughout these notes.

1.8.1 Arrays and Basic Operations

Def 1.27 (**NumPy Arrays**) The `ndarray` is the fundamental structure.

- **Creation:** `np.array([1, 2])`, `np.zeros((m, n))`, `np.random.randn(m, n)`.
- **Indexing:** 0-indexed. `A[i, j]` for entries; `A[i, :]` for rows; `A[:, j]` for columns.

Note 1.26 **Shapes and Rank-1 Arrays.** `A.shape` returns `(m, n)`. Note that a 1D array has shape `(n,)`, which is **not** equivalent to a column vector `(n, 1)`. These “rank-1 arrays” are a common source of broadcasting bugs. Use `x[:, None]` or `x.reshape(-1, 1)` to convert to 2D.

Ex 1.28

1. Create 4×4 identity matrix. Verify `shape` and `dtype`.
2. Extract row 2 and column 3 from a random 3×3 `A`.
3. Confirm that `np.zeros((3,))` fails where a `(3, 1)` column vector is required in a matrix-vector product.

1.8.2 Linear Algebra Operations

Core routines in `numpy.linalg`:

Function	Computes
<code>A @ B</code>	Matrix product $\mathbf{AB}$
<code>np.linalg.solve(A, b)</code>	Solve $\mathbf{Ax} = \mathbf{b}$ ($O(n^3)$)
<code>np.linalg.inv(A)</code>	Matrix inverse (Avoid; use <code>solve</code>)
<code>np.linalg.norm(x)</code>	Euclidean norm $\ \mathbf{x}\ _2$
<code>np.linalg.eig(A)</code>	Eigenvalues and eigenvectors
<code>np.linalg.svd(A)</code>	Singular Value Decomposition
<code>np.linalg.qr(A)</code>	QR Factorization
<code>np.linalg.cond(A)</code>	Condition number $\kappa_2(\mathbf{A})$

Note 1.27 Performance Law: Vectorization. Explicit `for` loops in Python are slow. Push heavy lifting into NumPy/BLAS routines by **vectorizing** operations. For example, use `A @ B` instead of triple-nested loops.

Note 1.28 Stability: solve vs inv. `np.linalg.solve(A, b)` is faster and more stable than `np.linalg.inv(A) @ b`. Explicit inversion is almost never desirable in scientific computation.

Ex 1.29

1. Solve a 2×2 system and verify the residual $\|\mathbf{Ax} - \mathbf{b}\|$.
2. Profile `solve` vs `inv @ b` for $n = 1000$.
3. Compare `np.linalg.eig` and `np.linalg.eigh` (optimized for symmetric matrices).

1.8.3 Key Idioms

- **Element-wise:** `*` and `**` are entry-wise. Matrix powers use `np.linalg.matrix_power`.
- **Broadcasting:** Automatic dimension expansion. `A * v` scales columns of `A` by entries of `v`.
- **Views vs. Copies:** `B = A` is a view (modifying `B` changes `A`). Use `B = A.copy()` for independent data. Slices are also views.

Ex 1.30

1. Let $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$. Compare `A * A` and `A @ A`.
2. Demonstrate the “view pitfall” by modifying a slice of a matrix.
3. Use broadcasting to perform diagonal scaling $\mathbf{AD}$ without constructing $\mathbf{D} = \text{diag}(d)$.

2 Direct Methods for Linear Systems

2.1 Linearity

Core definitions and structural properties of linear systems.

2.1.1 Linear Systems

A linear system encodes multiple equations into a single matrix-vector form.

Def 2.1 (Linear System) The matrix-vector form of a linear system is $\mathbf{Ax} = \mathbf{b}$, where:

- $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the coefficient matrix.
- $\mathbf{x} \in \mathbb{R}^n$ is the vector of unknowns.
- $\mathbf{b} \in \mathbb{R}^m$ is the right-hand side.

Ex 2.1

1. Convert $\{x + 2y = 3, 3x - y = 1\}$ to $\mathbf{Ax} = \mathbf{b}$.
2. Solve in NumPy via `np.linalg.solve(A, b)`. Verify with `np.linalg.norm(b - A @ x)`.

Thm 2.1 (Superposition Principle) If $\mathbf{Ax}_1 = \mathbf{b}_1$ and $\mathbf{Ax}_2 = \mathbf{b}_2$, then for any scalars α, β :

$$\mathbf{A}(\alpha\mathbf{x}_1 + \beta\mathbf{x}_2) = \alpha\mathbf{b}_1 + \beta\mathbf{b}_2.$$

Implication: Solutions to $\mathbf{Ax} = \mathbf{b}$ can be decomposed and recombined linearly.

Ex 2.2

1. Solve $\mathbf{Ax} = 3\mathbf{b}_1 - 2\mathbf{b}_2$ given only the solutions $\mathbf{x}_1, \mathbf{x}_2$ for $\mathbf{b}_1, \mathbf{b}_2$.
2. How does `np.linalg.solve(A, B)` for a matrix $\mathbf{B}$ exploit this?

2.1.2 Invertibility and Existence

For square systems ($\mathbf{A} \in \mathbb{R}^{n \times n}$), a unique solution exists iff $\mathbf{A}$ is nonsingular.

Thm 2.2 (Equivalences for Invertibility) The following are equivalent for $\mathbf{A} \in \mathbb{R}^{n \times n}$:

1. $\mathbf{A}$ is **invertible** ($\det(\mathbf{A}) \neq 0$).
2. $\mathbf{Ax} = \mathbf{b}$ has a **unique solution** for every $\mathbf{b}$.
3. $\mathbf{Ax} = \mathbf{0}$ has only the **trivial solution** ($\mathbf{x} = \mathbf{0}$).

4. $\text{rank}(\mathbf{A}) = n$ (Full Rank).
5. Columns of $\mathbf{A}$ are **linearly independent**.

Ex 2.3

1. Determine invertibility: $\mathbf{A} = \begin{pmatrix} 2 & -1 \\ 4 & -2 \end{pmatrix}$, $\mathbf{B} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}$.
2. Check consistency of $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 6 \end{pmatrix}$, $\mathbf{b} = \begin{pmatrix} 1 \\ 4 \end{pmatrix}$ via `np.linalg.matrix_rank`.

2.1.3 Linear Combinations and Bases

Note 2.1 The Column Perspective. The product $\mathbf{A}\mathbf{x}$ is a **linear combination of the columns of $\mathbf{A}$** weighted by the entries of $\mathbf{x}$:

$$\mathbf{A}\mathbf{x} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \cdots + x_n\mathbf{a}_n.$$

The system $\mathbf{A}\mathbf{x} = \mathbf{b}$ asks: “Can $\mathbf{b}$ be represented as a combination of $\mathbf{A}$ ’s columns?”

Def 2.2 (Linear Independence) Vectors $\{\mathbf{x}_1, \dots, \mathbf{x}_k\}$ are independent if $\sum \alpha_i \mathbf{x}_i = \mathbf{0} \Rightarrow \alpha_i = 0$ for all i .

Def 2.3 (Basis) A linearly independent set that spans the space. The number of vectors in any basis is the **dimension**.

2.1.4 Rank

Measures the number of independent directions (dimension of the column space).

Def 2.4 (Rank) $\text{rank}(\mathbf{A}) \leq \min(m, n)$.

- **Full Rank:** $\text{rank}(\mathbf{A}) = \min(m, n)$.
- **Singular:** Square matrix with $\text{rank}(\mathbf{A}) < n$.

Note 2.2 Numerical Rank. In code, never use row-reduction to find rank; it is numerically unstable. Use `np.linalg.matrix_rank`, which uses the Singular Value Decomposition (SVD).

Thm 2.3 (Rank-Nullity) For $\mathbf{A} \in \mathbb{R}^{m \times n}$: $\text{rank}(\mathbf{A}) + \text{nullity}(\mathbf{A}) = n$.

- **Nullity:** Dimension of the null space $\{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{0}\}$.

Ex 2.4

1. Find the general solution to a 2×3 system as $\mathbf{x} = \mathbf{x}_p + \mathbf{x}_n$ (particular + null space).
2. Why must a “wide” matrix ($m < n$) have a non-trivial null space?

2.1.5 The Four Fundamental Subspaces

Solution structure is characterized by four spaces defined by $\mathbf{A} \in \mathbb{R}^{m \times n}$ with rank r :

1. **Column Space** ($\mathcal{R}(\mathbf{A})$): $\mathbb{R}^m$, $\dim = r$. Span of columns.
2. **Null Space** ($\mathcal{N}(\mathbf{A})$): $\mathbb{R}^n$, $\dim = n - r$. Solutions to $\mathbf{A}\mathbf{x} = \mathbf{0}$.
3. **Row Space** ($\mathcal{R}(\mathbf{A}^T)$): $\mathbb{R}^n$, $\dim = r$. Span of rows.
4. **Left Null Space** ($\mathcal{N}(\mathbf{A}^T)$): $\mathbb{R}^m$, $\dim = m - r$. Solutions to $\mathbf{A}^T\mathbf{y} = \mathbf{0}$.

Thm 2.4 (Orthogonal Complements)

- $\mathcal{R}(\mathbf{A}^T) \perp \mathcal{N}(\mathbf{A})$ in $\mathbb{R}^n$.
- $\mathcal{R}(\mathbf{A}) \perp \mathcal{N}(\mathbf{A}^T)$ in $\mathbb{R}^m$.

Solvability: $\mathbf{A}\mathbf{x} = \mathbf{b}$ has a solution iff $\mathbf{b} \in \mathcal{R}(\mathbf{A})$, i.e., $\mathbf{b} \perp \mathcal{N}(\mathbf{A}^T)$.

Ex 2.5

1. For $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{pmatrix}$, find dimensions and bases for all four spaces.
2. Verify $\mathbf{b} = (1, 3)^T$ is not in the column space. What is the projection of $\mathbf{b}$ onto $\mathcal{R}(\mathbf{A})$?

2.2 LU Factorization

Solving $\mathbf{A}\mathbf{x} = \mathbf{b}$ via decomposition into triangular factors.

2.2.1 LU and Gaussian Elimination

The **LU factorization** expresses a square $\mathbf{A}$ as $\mathbf{A} = \mathbf{L}\mathbf{U}$, where $\mathbf{L}$ is unit lower triangular and $\mathbf{U}$ is upper triangular.

Def 2.5 (LU Factorization) $\mathbf{A} = \mathbf{L}\mathbf{U}$:

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & \dots \\ \ell_{21} & 1 & \dots \\ \vdots & \vdots & \ddots \end{pmatrix}, \quad \mathbf{U} = \begin{pmatrix} u_{11} & u_{12} & \dots \\ 0 & u_{22} & \dots \\ \vdots & \vdots & \ddots \end{pmatrix}.$$

Ex 2.6

1. Compare free parameters: how many in $\mathbf{L}$ vs. $\mathbf{U}$? Do they sum to n^2 ?
2. If $\mathbf{A} = \mathbf{L}\mathbf{U}$ is invertible, prove $\mathbf{L}$ and $\mathbf{U}$ are also invertible.

Note 2.3 Existence and Uniqueness. $\mathbf{A}$ admits a unique LU factorization iff all leading principal submatrices $\mathbf{A}_k$ are nonsingular ($k = 1, \dots, n - 1$). The algorithm fails if a zero pivot is encountered.

Note 2.4 Flop Cost. Computing $\mathbf{L}$ and $\mathbf{U}$ via Gaussian elimination costs $\frac{2}{3}n^3$ flops.

Note 2.5 The Solve Pattern. To solve $\mathbf{A}\mathbf{x} = \mathbf{b}$ given $\mathbf{A} = \mathbf{LU}$:

1. Solve $\mathbf{L}\mathbf{y} = \mathbf{b}$ via forward substitution ($O(n^2)$).
2. Solve $\mathbf{U}\mathbf{x} = \mathbf{y}$ via backward substitution ($O(n^2)$).

Advantage: Once $\mathbf{A}$ is factored, each new $\mathbf{b}$ costs only $O(n^2)$.

Ex 2.7

1. Factor $\mathbf{A} = \begin{pmatrix} 2 & 3 & 1 \\ 4 & 7 & 3 \\ -2 & -3 & 1 \end{pmatrix}$ by hand.
2. Solve $\mathbf{A}\mathbf{x} = (5, 15, 4)^T$ using your factors.
3. SciPy: Use `scipy.linalg.lu` and explain why it always returns a permutation $\mathbf{P}$.

2.2.2 Pivoting and Stability

Basic LU is numerically unstable if pivots are small. **Partial Pivoting** ensures stability by swapping rows to place the largest available entry on the diagonal.

Def 2.6 (LUP Decomposition) Recorded as $\mathbf{PA} = \mathbf{LU}$, where $\mathbf{P}$ is a permutation matrix.

Note 2.6 Stability and Growth Factor. Stability is controlled by the growth factor $\rho = \max |u_{ij}| / \max |a_{ij}|$. Partial pivoting keeps ρ small in practice (typically $O(n)$), ensuring backward stability. Theoretical worst-case is 2^{n-1} , but this is rarely seen.

Ex 2.8

1. Give a 2×2 matrix where LU fails but LUP succeeds.
2. Why is $\mathbf{P}^{-1}$ trivial to compute for any permutation matrix?
3. Complete pivoting (row and column swaps) exists but is rarely used. Why? (Cost vs. marginal stability gain).

2.2.3 Algorithm Cost Analysis

Note 2.7 Total Cost for k Right-Hand Sides.

$$\text{Cost} = \frac{2}{3}n^3 + 2kn^2.$$

Solving for 500 $\mathbf{b}$'s for $n = 1000$ using one factorization saves $\sim 500 \times \frac{2}{3}n^3$ flops compared to re-factorizing every time.

Ex 2.9

1. Sum the operations for forward substitution and show the cost is exactly $n^2 - n$.
2. Use `scipy.linalg.lu_factor` and `lu_solve` for a system with 3 right-hand sides. Verify residuals.

2.3 Symmetric Positive Definite (SPD) Matrices

Special class of matrices enabling highly efficient algorithms.

2.3.1 Definitions and Properties

SPD matrices arise in optimization (Hessians), statistics (covariances), and physical simulation (stiffness matrices).

Def 2.7 (Positive Definite) A symmetric matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is **positive definite** (PD) if:

$$\mathbf{x}^T \mathbf{A} \mathbf{x} > 0 \quad \text{for every nonzero } \mathbf{x} \in \mathbb{R}^n.$$

If $\mathbf{A}$ is both symmetric and PD, it is **SPD**.

Ex 2.10

1. Test $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ for PD property by expanding $\mathbf{x}^T \mathbf{A} \mathbf{x}$.
2. Prove a diagonal matrix $\mathbf{D}$ is PD iff all diagonal entries $d_i > 0$.

Note 2.8 Spectral Characterization. A symmetric matrix is PD iff **all its eigenvalues are strictly positive**. Every SPD matrix is invertible ($\det \mathbf{A} = \prod \lambda_i > 0$).

Ex 2.11

1. If $\mathbf{A}$ is SPD, show $\mathbf{B} = \mathbf{A} + c\mathbf{I}$ for $c > 0$ is also SPD.
2. Use `np.linalg.eigvalsh` to verify SPD property for several test matrices.

2.3.2 Geometry and Metric

An SPD matrix $\mathbf{A}$ defines a geometry on $\mathbb{R}^n$.

Def 2.8 (A-Inner Product) For SPD $\mathbf{A}$, $\langle \mathbf{x}, \mathbf{y} \rangle_{\mathbf{A}} = \mathbf{x}^T \mathbf{A} \mathbf{y}$ is a valid inner product.

- **A-Norm:** $\|\mathbf{x}\|_{\mathbf{A}} = \sqrt{\mathbf{x}^T \mathbf{A} \mathbf{x}}$.

- **Ellipsoid:** The set $\{\mathbf{x} : \|\mathbf{x}\|_{\mathbf{A}} = 1\}$ is an ellipse with axes along $\mathbf{A}$'s eigenvectors.

Note 2.9 **Physical Meaning:** Directions with large eigenvalues are “stiff” or “expensive” (small axes in the unit ellipsoid); small eigenvalues represent “sensitive” directions.

Ex 2.12

1. For $\mathbf{A} = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$, compute the distance between $(1, 0)^T$ and origin in both Euclidean and $\mathbf{A}$ -metrics.
2. Where does the $\mathbf{A}$ -metric appear in statistics? (Mahalanobis distance).

2.3.3 Cholesky Decomposition

The “LU equivalent” for SPD matrices. $\mathbf{A} = \mathbf{L}\mathbf{L}^T$.

Def 2.9 **(Cholesky Decomposition)** Every SPD matrix $\mathbf{A}$ admits a unique decomposition $\mathbf{A} = \mathbf{L}\mathbf{L}^T$, where $\mathbf{L}$ is lower triangular with strictly positive diagonal entries.

Note 2.10 **Flop Cost.** Cholesky costs $\frac{1}{3}n^3$ flops (half the price of LU).

Note 2.11 **Stability Advantage:** Unlike general LU, Cholesky is **guaranteed to be stable without pivoting**. In code, `np.linalg.cholesky` is the standard numerical test for positive definiteness; if it raises a `LinAlgError`, the matrix is not PD.

Ex 2.13

1. Factor $\mathbf{A} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$ by hand.
2. Why does Cholesky not require row swaps?
3. Solve $\mathbf{A}\mathbf{x} = \mathbf{b}$ using Cholesky for $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$.

Ex 2.14 **(Applications)**

1. **Covariance:** Generate $\mathbf{X} = \text{np.random.randn}(100, 4)$. Verify $\mathbf{X}^T \mathbf{X}$ is SPD via Cholesky.
2. **Gram Matrix:** Prove that $\mathbf{G} = \mathbf{X}^T \mathbf{X}$ is at least positive semi-definite for any $\mathbf{X}$, and PD if $\mathbf{X}$ has full column rank.

Ex 2.15 **(SPD Condition Number)** $\kappa_2(\mathbf{A}) = \lambda_{\max}/\lambda_{\min}$.

1. If $\mathbf{A}$ is SPD with eigenvalues $\{10, 0.01\}$, what is $\kappa_2(\mathbf{A})$?
2. Verify $\kappa_2(\mathbf{L}) = \sqrt{\kappa_2(\mathbf{A})}$ where $\mathbf{L}$ is the Cholesky factor.

3 Orthogonality and Least Squares

3.1 Orthogonal Matrices

Transformations that preserve lengths and angles. Numerically ideal due to optimal conditioning ($\kappa_2(\mathbf{Q}) = 1$).

3.1.1 Orthogonality and Orthonormal Bases

Def 3.1 (Orthogonal Vectors) $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ are orthogonal if $\mathbf{x}^T \mathbf{y} = 0$.

- **Mutually Orthogonal Set:** $\mathbf{v}_i^T \mathbf{v}_j = 0$ for all $i \neq j$.

Def 3.2 (Orthonormal Basis) A set $\{\mathbf{q}_1, \dots, \mathbf{q}_n\}$ where $\mathbf{q}_i^T \mathbf{q}_j = \delta_{ij}$ (unit length and mutually orthogonal).

- **Matrix Form:** If $\mathbf{Q} = [\mathbf{q}_1, \dots, \mathbf{q}_n]$, then $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$.

Note 3.1 **The Inner Product Advantage.** In an orthonormal basis, coefficients of $\mathbf{v} = \sum c_i \mathbf{q}_i$ are simple dot products: $c_i = \mathbf{q}_i^T \mathbf{v}$. This reduces solving a linear system ($O(n^3)$) to a sequence of dot products ($O(n^2)$).

Ex 3.1

1. Verify $\mathbf{q}_1 = \frac{1}{\sqrt{2}}(1, 1)^T, \mathbf{q}_2 = \frac{1}{\sqrt{2}}(1, -1)^T$ is an orthonormal basis for $\mathbb{R}^2$.
2. Express $\mathbf{v} = (3, -1)^T$ in this basis.
3. Prove that any set of n mutually orthogonal nonzero vectors is linearly independent.

Def 3.3 (Orthogonal Matrix) Square matrix $\mathbf{Q}$ where $\mathbf{Q}^T = \mathbf{Q}^{-1}$. Columns (and rows) form an orthonormal basis.

Thm 3.1 (Invariance Properties) For any orthogonal $\mathbf{Q}$ and vectors $\mathbf{x}, \mathbf{y}$:

1. **Inner Product Preserving:** $(\mathbf{Q}\mathbf{x})^T (\mathbf{Q}\mathbf{y}) = \mathbf{x}^T \mathbf{y}$.
2. **Isometry:** $\|\mathbf{Q}\mathbf{x}\|_2 = \|\mathbf{x}\|_2$.
3. **Conditioning:** $\kappa_2(\mathbf{Q}) = 1$ (Optimally conditioned).

Note 3.2 **Numerical Stability:** Because orthogonal matrices preserve norms, they do not amplify rounding errors. Transformations like Householder reflections are preferred over Gaussian elimination for high-stability requirements.

Ex 3.2

1. Verify rotation matrix $\mathbf{Q} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$ is orthogonal.
2. Prove that the product of two orthogonal matrices is orthogonal.
3. If $\mathbf{Q}$ is orthogonal, show $\det(\mathbf{Q}) = \pm 1$.

3.1.2 QR Decomposition

Factors $\mathbf{A}$ into an orthogonal $\mathbf{Q}$ and upper triangular $\mathbf{R}$. Default choice for least squares and eigenvalue algorithms.

Def 3.4 (QR Decomposition) For $\mathbf{A} \in \mathbb{R}^{m \times n}$ ($m \geq n$): $\mathbf{A} = \mathbf{QR}$.

- **Full QR:** $\mathbf{Q} \in \mathbb{R}^{m \times m}$, $\mathbf{R} \in \mathbb{R}^{m \times n}$.
- **Thin QR:** $\mathbf{Q}_1 \in \mathbb{R}^{m \times n}$, $\mathbf{R}_1 \in \mathbb{R}^{n \times n}$. Columns of $\mathbf{Q}_1$ span $\mathcal{R}(\mathbf{A})$.

Def 3.5 (Householder Reflector) A reflector $\mathbf{H} = \mathbf{I} - 2\frac{\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T\mathbf{v}}$ reflects $\mathbf{x}$ across the hyperplane $\mathbf{v}^\perp$.

- **Zeroing Property:** For any $\mathbf{x}$, we can choose $\mathbf{v}$ such that $\mathbf{H}\mathbf{x} = \pm\|\mathbf{x}\|\mathbf{e}_1$.
- **Efficiency:** Never form $\mathbf{H}$ explicitly. Apply as $\mathbf{H}\mathbf{x} = \mathbf{x} - \mathbf{v}(2\frac{\mathbf{v}^T\mathbf{x}}{\mathbf{v}^T\mathbf{v}})$, a rank-1 update costing $O(m)$.

Note 3.3 Flop Cost. For $\mathbf{A} \in \mathbb{R}^{m \times n}$ via Householder: Cost $\approx 2mn^2 - \frac{2}{3}n^3$. Square case ($m = n$): $\frac{4}{3}n^3$ (twice the cost of LU).

Ex 3.3

1. Find $\mathbf{v}$ s.t. $\mathbf{H}(3, 4)^T = (-5, 0)^T$.
2. Use Householder to triangularize $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 3 \\ 2 & 4 \end{pmatrix}$.
3. Compare thin vs. full QR shapes in NumPy for a 5×3 matrix.

3.1.3 Gram-Schmidt Process

Note 3.4 Classical vs. Modified Gram-Schmidt.

- **Classical (CGS):** $\mathbf{u}_k = \mathbf{v}_k - \sum_{j < k} \text{proj}_{\mathbf{u}_j}(\mathbf{v}_k)$. Numerically unstable for ill-conditioned $\mathbf{A}$ (Error $\propto \kappa(\mathbf{A})^2$).
- **Modified (MGS):** Orthogonalize sequentially against partially updated vectors. Better stability (Error $\propto \kappa(\mathbf{A})$).

Note 3.5 Numerical Hierarchy: Householder QR is the most stable ($O(\varepsilon_{\text{mach}})$ loss of orthogonality) and is used by `np.linalg.qr`. MGS is used when $\mathbf{Q}$ columns are needed one by one (e.g., Arnoldi/GMRES).

Ex 3.4

1. Apply hand CGS to columns of $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 0 & 1 \end{pmatrix}$.
2. **Stability Experiment:** Construct a nearly singular Vandermonde matrix. Compare loss of orthogonality ($\|\mathbf{Q}^T \mathbf{Q} - \mathbf{I}\|_F$) for CGS, MGS, and Householder.

3.1.4 Orthogonal Projection

Def 3.6 (Orthogonal Projector) For a subspace S with orthonormal basis $\mathbf{Q}$: $\mathbf{P} = \mathbf{Q}\mathbf{Q}^T$.

- **Properties:** $\mathbf{P}^2 = \mathbf{P}$ (Idempotent), $\mathbf{P}^T = \mathbf{P}$ (Symmetric).
- **Geometric View:** $\mathbf{P}\mathbf{x}$ is the closest vector in S to $\mathbf{x}$. Residual $\mathbf{r} = \mathbf{x} - \mathbf{P}\mathbf{x}$ lies in $S^\perp = \mathcal{N}(\mathbf{Q}^T)$.

Ex 3.5

1. Show that for any projector $\mathbf{P}$, the residual $\mathbf{x} - \mathbf{P}\mathbf{x}$ is orthogonal to $\mathbf{P}\mathbf{x}$.
2. Project $\mathbf{x} = (1, 2, 3)^T$ onto the span of $(1, 1, 1)^T$.

3.2 Vector and Matrix Norms

Foundations for measuring approximation errors and convergence.

3.2.1 Vector Norms

Def 3.7 (Vector Norm) $\|\cdot\| : \mathbb{R}^n \rightarrow \mathbb{R}$ satisfies:

1. **Definiteness:** $\|\mathbf{x}\| \geq 0$, with equality iff $\mathbf{x} = \mathbf{0}$.
2. **Homogeneity:** $\|c\mathbf{x}\| = |c|\|\mathbf{x}\|$.
3. **Triangle Inequality:** $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$.

Def 3.8 (ℓ^p Norms) $\|\mathbf{x}\|_p = (\sum |x_i|^p)^{1/p}$.

- ℓ_1 : $\sum |x_i|$ (Sparsity).
- ℓ_2 : $\sqrt{\mathbf{x}^T \mathbf{x}}$ (Energy).

- ℓ_∞ : $\max |x_i|$ (Tolerance).

Thm 3.2 (Fundamental Inequalities)

- **Cauchy-Schwarz:** $|\mathbf{x}^T \mathbf{y}| \leq \|\mathbf{x}\|_2 \|\mathbf{y}\|_2$.
- **Hölder:** $|\mathbf{x}^T \mathbf{y}| \leq \|\mathbf{x}\|_p \|\mathbf{y}\|_q$ where $1/p + 1/q = 1$.

Note 3.6 Equivalence: On $\mathbb{R}^n$, all norms are equivalent: $c_1 \|\mathbf{x}\|_\beta \leq \|\mathbf{x}\|_\alpha \leq c_2 \|\mathbf{x}\|_\beta$. Convergence in one implies convergence in all.

Ex 3.6

1. Prove the triangle inequality for $\|\cdot\|_2$ using Cauchy-Schwarz.
2. Show that $\|\mathbf{x}\|_\infty = \lim_{p \rightarrow \infty} \|\mathbf{x}\|_p$.

3.2.2 Matrix Norms

Measure how much a matrix amplifies vectors.

Def 3.9 (Induced (Operator) Norm) $\|\mathbf{A}\| = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbf{A}\mathbf{x}\|}{\|\mathbf{x}\|}$.

- $\|\mathbf{A}\|_1$: Max absolute column sum.
- $\|\mathbf{A}\|_\infty$: Max absolute row sum.
- $\|\mathbf{A}\|_2$ (**Spectral**): $\sigma_{\max}(\mathbf{A}) = \sqrt{\lambda_{\max}(\mathbf{A}^T \mathbf{A})}$.

Def 3.10 (Frobenius Norm) $\|\mathbf{A}\|_F = \sqrt{\sum a_{ij}^2} = \sqrt{\text{Tr}(\mathbf{A}^T \mathbf{A})}$.

- **Relation:** $\|\mathbf{A}\|_2 \leq \|\mathbf{A}\|_F \leq \sqrt{\text{rank}(\mathbf{A})} \|\mathbf{A}\|_2$.

Note 3.7 Consistency and Submultiplicativity: Matrix norms are useful when they satisfy $\|\mathbf{A}\mathbf{x}\| \leq \|\mathbf{A}\| \|\mathbf{x}\|$. All induced norms are additionally submultiplicative: $\|\mathbf{A}\mathbf{B}\| \leq \|\mathbf{A}\| \|\mathbf{B}\|$.

Ex 3.7

1. Compute $\|\mathbf{A}\|_1, \|\mathbf{A}\|_\infty, \|\mathbf{A}\|_2, \|\mathbf{A}\|_F$ for $\mathbf{A} = \begin{pmatrix} 1 & -2 \\ -3 & 4 \end{pmatrix}$.
2. **Low-Rank Approximation:** The Frobenius norm is central to the Eckart-Young-Mirsky theorem. How is it related to singular values?
3. Find a 2×2 example where the max norm ($\max |a_{ij}|$) is not submultiplicative.

3.3 Stability and Condition Number

Sensitivity of problems vs. robustness of algorithms.

3.3.1 Why Factorize instead of Invert?

Note 3.8 **Rule:** Never compute $\mathbf{A}^{-1}$ explicitly.

- **Cost:** Computing $\mathbf{A}^{-1}$ takes $3\times$ longer than LU ($\frac{8}{3}n^3$ vs. $\frac{2}{3}n^3$).
- **Stability:** Round-off in $\mathbf{A}^{-1}$ propagates into the entire product $\mathbf{A}^{-1}\mathbf{b}$. Stable factorizations confine errors.
- **Efficiency:** LU allows solving for new $\mathbf{b}$'s in $O(n^2)$ without storing a dense inverse.

Ex 3.8

1. For $n = 500$, compare total flops for solving 10 systems via $\mathbf{A}^{-1}$ vs. LU.
2. Construct $\mathbf{A} = \text{diag}(1, 1, 10^{-12})$. Compare residuals of `inv(A) @ b` and `solve(A, b)`.

3.3.2 Numerical Stability

A property of the **algorithm**.

Def 3.11 (**Backward Stability**) An algorithm is backward stable if its computed $\hat{\mathbf{x}}$ is the **exact solution to a nearby problem**: $(\mathbf{A} + \delta\mathbf{A})\hat{\mathbf{x}} = \mathbf{b} + \delta\mathbf{b}$, with $\|\delta\mathbf{A}\|/\|\mathbf{A}\| = O(\epsilon_{\text{mach}})$.

Note 3.9 **GEPP Stability:** Gaussian Elimination with Partial Pivoting is backward stable for virtually all practical matrices. The growth factor ρ stays small, preventing exponential error amplification.

Ex 3.9

1. Explain why a small residual doesn't always guarantee a small error (Forward Error).
2. Histogram growth factors for 100 random matrices via `scipy.linalg.lu`.

3.3.3 Condition Number

A property of the **problem**.

Def 3.12 (**Condition Number**) $\kappa(\mathbf{A}) = \|\mathbf{A}\|\|\mathbf{A}^{-1}\|$.

- **Interpretation:** Worst-case error amplification. A perturbation of size ϵ in $\mathbf{b}$ can cause error up to $\kappa(\mathbf{A})\epsilon$ in $\mathbf{x}$.

- **Rule of Thumb:** You expect at most $16 - \log_{10}(\kappa(\mathbf{A}))$ correct digits in double precision.

Ex 3.10

1. Compute $\kappa_2(H_n)$ for Hilbert matrices $n = 3, \dots, 10$. How fast does it grow?
2. **The Neumann Series:** Use it to prove the perturbation bound: $\frac{\|\delta \mathbf{x}\|}{\|\mathbf{x}\|} \leq \frac{\kappa(\mathbf{A})}{1 - \dots}$.

3.3.4 Stability Hierarchy

The “Golden Rule” for choosing solvers:

1. **SVD:** Most stable, most expensive ($O(n^3)$ with large constant).
2. **Householder QR:** Very stable, no growth factor ($O(mn^2)$).
3. **Cholesky:** Stable for SPD systems, half the cost of LU.
4. **GEPP (LU with Pivoting):** Standard for general systems.
5. **LU without Pivoting:** Dangerously unstable; avoid.

Def 3.13 (Iterative Refinement) “Polish” a solution $\hat{\mathbf{x}}$ by:

1. $\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$
2. Solve $\mathbf{A}\boldsymbol{\delta} = \mathbf{r}$
3. $\hat{\mathbf{x}} \leftarrow \hat{\mathbf{x}} + \boldsymbol{\delta}$

Each step recovers digits lost due to conditioning, up to machine precision.

Ex 3.11

1. Solve $H_{10}\mathbf{x} = H_{10}\mathbf{1}$. Apply one step of iterative refinement. How many digits are recovered?
2. Compare forward errors of `solve` (LU) vs `lstsq` (QR) on a Hilbert system.

3.4 Overdetermined Systems and Least Squares

Finding the “best-fit” solution when $m > n$ equations and no exact solution exists.

3.4.1 Least Squares Problem

Def 3.14 (Overdetermined System) $\mathbf{Ax} = \mathbf{b}$ with $\mathbf{A} \in \mathbb{R}^{m \times n}$, $m > n$. Usually $\mathbf{b} \notin \mathcal{R}(\mathbf{A})$.

Def 3.15 (Least Squares Problem) Find $\hat{\mathbf{x}}$ that minimizes the Euclidean residual:

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|_2^2.$$

Geometry: $\mathbf{A}\hat{\mathbf{x}}$ is the orthogonal projection of $\mathbf{b}$ onto $\mathcal{R}(\mathbf{A})$.

3.4.2 Normal Equations

Thm 3.3 (Normal Equations) The minimizer satisfies $\mathbf{A}^T \mathbf{A} \hat{\mathbf{x}} = \mathbf{A}^T \mathbf{b}$.

- **Condition:** $\mathbf{A}^T \mathbf{r} = \mathbf{0}$ (Residual must be normal to the column space).
- **Unique Solution:** $\hat{\mathbf{x}} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$ if $\mathbf{A}$ has full column rank.

Note 3.10 **The Stability Trap:** $\kappa_2(\mathbf{A}^T \mathbf{A}) = \kappa_2(\mathbf{A})^2$. Squaring the condition number can destroy precision. If $\kappa_2(\mathbf{A}) = 10^6$, you may lose 12 digits of accuracy. **Practical Rule:** Avoid normal equations for ill-conditioned matrices.

Ex 3.12

1. Fit a line $y = mx + b$ to points $(1, 2), (2, 3), (3, 5), (4, 4)$ via normal equations.
2. Prove $\kappa_2(\mathbf{A}^T \mathbf{A}) = \kappa_2(\mathbf{A})^2$ using singular values.

3.4.3 QR-Based Least Squares

The numerically superior approach.

Thm 3.4 (Solving via QR) If $\mathbf{A} = \mathbf{QR}$ (thin QR), the least squares solution satisfies:

$$\mathbf{R}\hat{\mathbf{x}} = \mathbf{Q}^T \mathbf{b}.$$

Why? $\|\mathbf{Ax} - \mathbf{b}\|_2^2 = \|\mathbf{Rx} - \mathbf{Q}^T \mathbf{b}\|_2^2 + \|(\mathbf{I} - \mathbf{QQ}^T)\mathbf{b}\|_2^2$. The first term is zeroed by the solve.

Note 3.11 **Performance:** QR-based least squares costs $O(mn^2)$. It is the default in `np.linalg.lstsq`.

Ex 3.13

1. Solve the line-fitting problem using QR. Compare coefficients with the normal equations result.
2. **Regularization:** If $\mathbf{A}$ is rank-deficient, $\|\hat{\mathbf{x}}\|$ can explode. Add a penalty term $\lambda \|\mathbf{x}\|_2^2$ (Ridge Regression). How do the normal equations change?

3.4.4 Polynomial Regression

Def 3.16 (Vandermonde Matrix) For nodes $t_1, \dots, t_m$, the degree- n design matrix is:

$$V_{ij} = t_i^{j-1}.$$

Polynomial fitting $p(t) = \sum c_j t^j$ is solved as $\mathbf{V}\mathbf{c} \approx \mathbf{f}$.

Ex 3.14

1. Fit a degree-2 polynomial to 6 data points. Verify the residual and R^2 score.
2. **Numerical Stability:** Construct a Vandermonde matrix with $n = 10$. Compare the errors of `lstsq(A, b)` vs `solve(A.T @ A, A.T @ b)`.

3.5 Chebyshev Polynomials

Minimax-optimal basis for polynomial approximation. Resolves ill-conditioning and oscillations of the monomial basis.

3.5.1 Failure of the Monomial Basis

1. **Numerical:** Vandermonde matrices $V_{ij} = x_i^{j-1}$ have condition numbers that grow exponentially with degree.
2. **Analytical: Runge's Phenomenon.** Interpolating smooth functions at equally spaced nodes leads to divergence at boundaries as degree $n \rightarrow \infty$.

Ex 3.15

1. Build $n = 15$ Vandermonde matrix on equispaced nodes. Check $\kappa_2(\mathbf{V})$.
2. Plot Runge's function $f(x) = 1/(1+25x^2)$ and its $n = 10$ interpolant. Note boundary oscillations.

3.5.2 Chebyshev Polynomials (T_k)

Defined on $[-1, 1]$ by $T_k(x) = \cos(k \arccos x)$.

Def 3.17 (Properties of T_k)

- **Recurrence:** $T_0 = 1, T_1 = x, T_{k+1} = 2xT_k - T_{k-1}$.
- **Boundedness:** $|T_k(x)| \leq 1$ for all $x \in [-1, 1]$.
- **Orthogonality:** $\int_{-1}^1 T_j T_k \frac{dx}{\sqrt{1-x^2}} = 0$ for $j \neq k$.

Ex 3.16

1. Compute T_3, T_4 via recurrence.
2. Plot $T_0, \dots, T_5$. Note they all oscillate between ± 1 .

3.5.3 Chebyshev Nodes

Zeros of $T_n(x)$ provide the optimal node distribution for interpolation.

Def 3.18 (Chebyshev Nodes) $x_k = \cos(\frac{(2k-1)\pi}{2n})$ for $k = 1, \dots, n$.

- **Node Clustering:** Nodes cluster near boundaries ± 1 . This suppresses Runge's oscillations.

Note 3.12 The Minimax Property: Among all monic polynomials of degree n , $\tilde{T}_n = T_n/2^{n-1}$ has the smallest possible maximum value on $[-1, 1]$. Using these nodes minimizes the interpolation error bound.

Ex 3.17

1. Plot 10 Chebyshev nodes vs. 10 equispaced nodes. Note clustering.
2. Repeat the Runge function experiment with Chebyshev nodes. Observe convergence.
3. Compare κ_2 of Vandermonde vs. Chebyshev basis ($V_{ij} = T_{j-1}(x_i)$).

3.5.4 Chebyshev Series

Truncated expansions $f(x) \approx \sum c_k T_k(x)$.

Thm 3.5 (Spectral Accuracy) If f is analytic on $[-1, 1]$, the Chebyshev coefficients c_k decay **exponentially**. Truncation error $\|f - p_n\|_\infty = O(\rho^{-n})$. **Insight:** Just 20–30 terms often yield machine precision.

Ex 3.18

1. Compute degree- n fit to e^x via `np.polynomial.chebyshev.chebfit`. Plot error vs. n .
2. **Spectral Differentiation:** The derivative of a Chebyshev series satisfies a simple recurrence. Compare `chebder` accuracy to finite differences for $f(x) = \sin(\pi x)$.

4 Eigenvalues and the SVD

4.1 Eigenvalues and Eigenvectors

Reveals the scaling directions of a linear transformation. Essential for stability analysis, PCA, and spectral methods.

4.1.1 Definitions and Basic Properties

Def 4.1 (Eigenvalues and Eigenvectors) For $\mathbf{A} \in \mathbb{R}^{n \times n}$, a scalar $\lambda \in \mathbb{C}$ is an **eigenvalue** if $\exists$ nonzero $\mathbf{v} \in \mathbb{C}^n$ such that:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}.$$

- **Spectrum** $\sigma(\mathbf{A})$: The set of all eigenvalues.
- **Invariance**: Multiplying $\mathbf{v}$ by $\mathbf{A}$ only scales its length; its direction remains fixed.

Def 4.2 (Characteristic Polynomial) $p_{\mathbf{A}}(\lambda) = \det(\mathbf{A} - \lambda\mathbf{I})$. Eigenvalues are the roots of $p_{\mathbf{A}}(\lambda) = 0$.

Ex 4.1

1. Verify $\mathbf{v} = (1, 1)^T$ is an eigenvector of $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$ and find λ .
2. Why is the zero vector excluded from the definition of an eigenvector?

Thm 4.1 (Similar Matrices) If $\mathbf{B} = \mathbf{V}^{-1}\mathbf{A}\mathbf{V}$ (Similarity Transform), then $\sigma(\mathbf{B}) = \sigma(\mathbf{A})$. **Insight**: Eigenvalues are invariant under change of basis.

Thm 4.2 (Key Properties)

1. **Trace**: $\sum \lambda_i = \text{tr}(\mathbf{A})$.
2. **Determinant**: $\prod \lambda_i = \det(\mathbf{A})$.
3. **Singularity**: $0 \in \sigma(\mathbf{A}) \iff \mathbf{A}$ is singular.
4. **Inversion**: If $\mathbf{A}$ is invertible, eigenvalues of $\mathbf{A}^{-1}$ are $1/\lambda_i$.

4.1.2 Symmetric and Hermitian Matrices

The “Golden State” of eigenvalue problems.

Thm 4.3 (Spectral Theorem (Real Symmetric)) If $\mathbf{A} = \mathbf{A}^T$:

1. All eigenvalues λ_i are **real**.
2. Eigenvectors for distinct λ are **orthogonal**.
3. $\mathbf{A}$ is **orthogonally diagonalizable**: $\mathbf{A} = \mathbf{Q}\mathbf{D}\mathbf{Q}^T$.

Def 4.3 (Hermitian Matrix) $\mathbf{A} = \mathbf{A}^*$ (conjugate transpose). The complex analog of symmetric matrices.

- **Spectral Theorem (Hermitian):** Unitarily diagonalizable ($\mathbf{A} = \mathbf{Q}\mathbf{D}\mathbf{Q}^*$) with real eigenvalues.

Ex 4.2

1. Prove eigenvalues of $\mathbf{A}^2$ are λ_i^2 .
2. Diagonalize $\mathbf{A} = \begin{pmatrix} 4 & 1 \\ 1 & 4 \end{pmatrix}$ by finding an orthonormal eigenbasis.

4.1.3 Iterative Algorithms

In practice, we compute eigenvalues via iteration, not by finding roots of $p_{\mathbf{A}}(\lambda)$.

Def 4.4 (Power Iteration) Repeatedly apply $\mathbf{A}$ to a unit vector: $\mathbf{v}^{(k)} = \frac{\mathbf{A}\mathbf{v}^{(k-1)}}{\|\mathbf{A}\mathbf{v}^{(k-1)}\|}$.

- **Convergence:** $\mathbf{v}^{(k)} \rightarrow$ dominant eigenvector (for $|\lambda_1| > |\lambda_2|$).
- **Rate:** $O(|\lambda_2/\lambda_1|^k)$.

Def 4.5 (Inverse Iteration) Apply power iteration to $(\mathbf{A} - \sigma\mathbf{I})^{-1}$.

- **Use:** Finds the eigenvalue closest to the shift σ .
- **Efficiency:** Factorize $(\mathbf{A} - \sigma\mathbf{I})$ once, then solve at each step ($O(n^2)$).

Def 4.6 (Rayleigh Quotient Iteration (RQI)) Update the shift σ_k to the current Rayleigh quotient $(\mathbf{v}^{(k)})^T \mathbf{A} \mathbf{v}^{(k)}$ at each step.

- **Convergence:** Cubic for symmetric matrices (digits triple each step).

Ex 4.3

1. Implement power iteration to find $\lambda_{\max}$ of a tridiagonal matrix.
2. Use RQI on a random 4×4 symmetric matrix. Observe the rate of convergence.

4.1.4 Schur Decomposition and the QR Algorithm

The theoretical and practical backbone of general eigensolvers.

Thm 4.4 (Schur Decomposition) Every square $\mathbf{A}$ satisfies $\mathbf{A} = \mathbf{Q}\mathbf{T}\mathbf{Q}^*$, where $\mathbf{Q}$ is unitary and $\mathbf{T}$ is upper triangular.

- **Schur Vectors:** Columns of $\mathbf{Q}$ provide an orthonormal basis for nested invariant subspaces.

Def 4.7 (QR Iteration) Iteratively compute QR factorizations and swap the factors:

1. $\mathbf{A}_k = \mathbf{Q}_k \mathbf{R}_k$
2. $\mathbf{A}_{k+1} = \mathbf{R}_k \mathbf{Q}_k = \mathbf{Q}_k^T \mathbf{A}_k \mathbf{Q}_k$

Convergence: $\mathbf{A}_k$ converges to the Schur form $\mathbf{T}$.

Note 4.1 Performance: Raw QR iteration is $O(n^3)$ per step. Practical implementations (like `np.linalg.eig`) use:

1. **Hessenberg Reduction:** Pre-process $\mathbf{A}$ to upper Hessenberg form ($O(n^3)$ once).
2. **Triangularization:** Hessenberg QR steps cost only $O(n^2)$.
3. **Shifts:** Accelerate convergence to quadratic or cubic.

Ex 4.4

1. For $\mathbf{A} = \begin{pmatrix} 3 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 2 \end{pmatrix}$, identify the Schur vectors and eigenvalues by inspection.
2. Implement 50 steps of basic QR iteration on a random 3×3 symmetric matrix. Check the off-diagonals.

4.2 Singular Value Decomposition

The “Master Decomposition” of linear algebra. Decomposes any matrix into orthogonal transformations and diagonal scaling.

4.2.1 The SVD Theorem

Thm 4.5 (Singular Value Decomposition (SVD)) For $\mathbf{A} \in \mathbb{R}^{m \times n}$, there exist orthogonal matrices $\mathbf{U} \in \mathbb{R}^{m \times m}$, $\mathbf{V} \in \mathbb{R}^{n \times n}$ and diagonal $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ such that:

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T.$$

- **Singular Values (σ_i):** Non-negative, ordered $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$.
- **Modes:** $\mathbf{A} = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T$ (Outer product expansion).

Note 4.2 Geometric Interpretation: $\mathbf{A}$ rotates the input space ($\mathbf{V}^T$), scales along the axes ($\mathbf{\Sigma}$), and rotates the result to the output space ($\mathbf{U}$).

4.2.2 The Four Fundamental Subspaces

The SVD provides optimal orthonormal bases for the spaces defined by $\mathbf{A}$ (rank r):

1. **Column Space** ($\mathcal{R}(\mathbf{A})$): First r columns of $\mathbf{U}$.
2. **Null Space** ($\mathcal{N}(\mathbf{A})$): Last $n - r$ columns of $\mathbf{V}$.
3. **Row Space** ($\mathcal{R}(\mathbf{A}^T)$): First r columns of $\mathbf{V}$.
4. **Left Null Space** ($\mathcal{N}(\mathbf{A}^T)$): Last $m - r$ columns of $\mathbf{U}$.

Thm 4.6 (Relationship to Eigendecomposition)

- Right singular vectors ($\mathbf{v}_i$) are eigenvectors of $\mathbf{A}^T \mathbf{A}$.
- Left singular vectors ($\mathbf{u}_i$) are eigenvectors of $\mathbf{A} \mathbf{A}^T$.
- Singular values $\sigma_i = \sqrt{\lambda_i(\mathbf{A}^T \mathbf{A})}$.

4.2.3 Low-Rank Approximation

Thm 4.7 (Eckart-Young-Mirsky Theorem) The best rank- k approximation ($k < r$) of $\mathbf{A}$ in spectral and Frobenius norms is the truncated SVD:

$$\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T.$$

- **Error:** $\|\mathbf{A} - \mathbf{A}_k\|_2 = \sigma_{k+1}$.

Note 4.3 Applications: Truncated SVD is the engine behind Principal Component Analysis (PCA), image compression, and latent semantic analysis.

4.2.4 The Pseudoinverse

Def 4.8 (Moore-Penrose Pseudoinverse) $\mathbf{A}^+ = \mathbf{V} \mathbf{\Sigma}^+ \mathbf{U}^T$, where $(\mathbf{\Sigma}^+)_{ii} = 1/\sigma_i$ if $\sigma_i > 0$, else 0.

- **Minimum-Norm Solution:** For any system $\mathbf{A} \mathbf{x} = \mathbf{b}$, the vector $\hat{\mathbf{x}} = \mathbf{A}^+ \mathbf{b}$ is the shortest vector minimizing $\|\mathbf{A} \mathbf{x} - \mathbf{b}\|_2$.

Ex 4.5

1. Compute the SVD of $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix}$ by hand.

2. **Compression:** Load a 512×512 image. Plot the relative error $\|\mathbf{A} - \mathbf{A}_k\|_F / \|\mathbf{A}\|_F$ vs. k . At what k is the image recognizable?
3. Use the pseudoinverse to find the least squares solution to a rank-deficient 3×3 system.

4.3 Lanczos Algorithm

Iterative method for finding extreme eigenpairs of large, symmetric matrices without explicit $O(n^2)$ formation.

4.3.1 Krylov Subspaces

Def 4.9 (Krylov Subspace) $\mathcal{K}_k(\mathbf{A}, \mathbf{v}) = \text{span}\{\mathbf{v}, \mathbf{A}\mathbf{v}, \mathbf{A}^2\mathbf{v}, \dots, \mathbf{A}^{k-1}\mathbf{v}\}$. **Intuition:** $\mathbf{A}^j\mathbf{v}$ amplifies components along dominant eigenvectors, concentrating information about the spectral edges.

4.3.2 The Lanczos Recurrence

Thm 4.8 (Lanczos Relation) The process builds an orthonormal basis $V_k = [\mathbf{v}_1, \dots, \mathbf{v}_k]$ for $\mathcal{K}_k$ satisfying:

$$\mathbf{A}V_k = V_k T_k + \beta_k \mathbf{v}_{k+1} \mathbf{e}_k^T.$$

- T_k : Symmetric **tridiagonal** matrix.
- $\alpha_j = \mathbf{v}_j^T \mathbf{A} \mathbf{v}_j$: Rayleigh quotients (diagonal).
- $\beta_j = \|\text{residual}\|$: Orthogonalization coefficients (off-diagonal).

Note 4.4 Performance Advantage: Unlike Gram-Schmidt, which orthogonalizes against all previous vectors, Lanczos only requires the **two previous vectors** (three-term recurrence). This makes it $O(k \cdot \text{nnz})$ for sparse matrices.

4.3.3 Ritz Values and Vectors

Approximations to the true eigenpairs from the Krylov subspace.

Def 4.10 (Ritz Pair)

- **Ritz Value** (θ_i): Eigenvalue of T_k .
- **Ritz Vector** ($\tilde{\mathbf{u}}_i$): $V_k \mathbf{s}_i$, where $\mathbf{s}_i$ is an eigenvector of T_k .

Thm 4.9 (Residual Estimate) The residual $\|\mathbf{A}\tilde{\mathbf{u}}_i - \theta_i \tilde{\mathbf{u}}_i\|_2 = \beta_k |s_i(k)|$. **Significance:** Convergence of a Ritz pair can be monitored for $O(k)$ cost without forming the full n -vector $\tilde{\mathbf{u}}_i$.

Note 4.5 Ghost Eigenvalues: In floating-point, the three-term recurrence loses orthogonality, leading to “ghost” (spurious) copies of eigenvalues. **Practical Rule:** Use full or selective reorthogonalization to maintain basis integrity.

Ex 4.6

1. Carry out 2 steps of Lanczos for $\mathbf{A} = \text{diag}(5, 3, 1)$ starting with $\mathbf{v}_1 = \frac{1}{\sqrt{3}}\mathbf{1}$.
2. Show that $T_k = V_k^T \mathbf{A} V_k$ (Galerkin Projection).
3. Implement Lanczos on a 50×50 Hilbert matrix. Count spurious eigenvalues without reorthogonalization.

4.4 Jacobi Eigenvalue Algorithm

Iterative algorithm for symmetric matrices using plane rotations to zero off-diagonal entries.

4.4.1 Givens Rotations

Def 4.11 (Givens Rotation) $G(p, q, \theta)$ is an identity matrix with a 2×2 rotation block at $(p, p), (p, q), (q, p), (q, q)$.

- **Similarity Transform:** $\mathbf{A}' = G^T \mathbf{A} G$.
- **Effect:** Modifies only rows/columns p and q . Preserves eigenvalues ($\mathbf{A}'$ is orthogonally similar to $\mathbf{A}$).

Thm 4.10 (Optimal Rotation Angle) To set $A'_{pq} = 0$, let $\tau = (A_{qq} - A_{pp})/(2A_{pq})$. The optimal tangent $t = \tan \theta$ is:

$$t = \frac{\text{sign}(\tau)}{|\tau| + \sqrt{1 + \tau^2}}, \quad c = \frac{1}{\sqrt{1 + t^2}}, \quad s = ct.$$

Choosing the smaller root $|t| \leq 1$ ensures numerical stability and minimal perturbation.

4.4.2 Convergence and Cost

Thm 4.11 (Monotone Convergence) Let $\sigma_{\text{off}}^2(\mathbf{A}) = \sum_{p < q} A_{pq}^2$. Each rotation reduces this norm:

$$\sigma_{\text{off}}^2(\mathbf{A}') = \sigma_{\text{off}}^2(\mathbf{A}) - A_{pq}^2.$$

Convergence: Off-diagonals decrease monotonically. Convergence is ultimately **quadratic** (errors square each sweep).

Note 4.6 Performance vs. Accuracy:

- **Cost:** $O(n^3)$ per sweep. Typically 5–15 sweeps for convergence. More expensive than QR.
- **Accuracy:** Unlike QR, Jacobi achieves **high relative accuracy** on small eigenvalues.

Ex 4.7

1. Apply one Jacobi rotation to zero A_{12} in $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$.
2. Show that after zeroing A_{pq} , the new diagonal entries A'_{pp} and A'_{qq} are the eigenvalues of the 2×2 submatrix at $\{p, q\}$.
3. Perform one full cyclic sweep on $\mathbf{A} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 3 \end{pmatrix}$ by hand.

5 Iterative Methods for Linear Systems

5.1 Iterative Methods for Linear Systems

Alternative to direct solvers for large, sparse systems where matrix factorization is impractical due to memory or compute limits.

5.1.1 Direct vs. Iterative Solvers

Note 5.1 The Sparsity Problem: Even if $\mathbf{A}$ is sparse, its LU or Cholesky factors are typically much denser (**fill-in**). For a 2D Poisson problem on an $n \times n$ grid:

- $\mathbf{A}$ has $O(n^2)$ nonzeros.
- LU factors have $O(n^3)$ nonzeros.

Iterative methods exploit sparsity by using only matrix-vector products ($O(\text{nnz})$).

Def 5.1 (Stationary Iteration) An iteration of the form $\mathbf{x}^{(k+1)} = \mathbf{T}\mathbf{x}^{(k)} + \mathbf{c}$, where:

- $\mathbf{T} = \mathbf{I} - \mathbf{M}^{-1}\mathbf{A}$ is the iteration matrix.
- $\mathbf{c} = \mathbf{M}^{-1}\mathbf{b}$ is the updated right-hand side.
- $\mathbf{M}$ is the **splitting matrix** (ideally $\mathbf{M} \approx \mathbf{A}$ and $\mathbf{M}^{-1}$ is cheap).

5.1.2 Convergence and Spectral Radius

Def 5.2 (Spectral Radius) $\rho(\mathbf{B}) = \max\{|\lambda| : \lambda \in \sigma(\mathbf{B})\}$.

Thm 5.1 (Gelfand's Formula) For any submultiplicative norm, $\rho(\mathbf{B}) = \lim_{k \rightarrow \infty} \|\mathbf{B}^k\|^{1/k}$. **Insight:** For large k , $\|\mathbf{B}^k\| \approx \rho(\mathbf{B})^k$.

Thm 5.2 (Fundamental Convergence Condition) The iteration $\mathbf{x}^{(k+1)} = \mathbf{T}\mathbf{x}^{(k)} + \mathbf{c}$ converges for any $\mathbf{x}^{(0)}$ iff $\rho(\mathbf{T}) < 1$.

- **Error Evolution:** $\mathbf{e}^{(k)} = \mathbf{T}^k \mathbf{e}^{(0)}$.
- **Rate:** $\|\mathbf{e}^{(k)}\| \approx \rho(\mathbf{T})^k \|\mathbf{e}^{(0)}\|$.

Note 5.2 **Practical Cost:** To reduce error by 10^{-p} , the required iterations are $k \approx \frac{p}{\log_{10}(1/\rho(\mathbf{T}))}$. As $\rho(\mathbf{T}) \rightarrow 1^-$, the cost explodes.

Ex 5.1

1. For $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, find $\rho(\mathbf{T})$ for the splitting $\mathbf{M} = 3\mathbf{I}$.
2. If $\rho(\mathbf{T}) = 0.99$, how many iterations are needed to gain 4 digits of precision?
3. Start $\mathbf{x}^{(0)} = \mathbf{0}$ and solve $\mathbf{A}\mathbf{x} = (4, 4)^T$ via 3 steps of this iteration.

5.1.3 Fill-in and Sparse Storage

Ex 5.2

1. **Fill-in Demo:** Generate a large sparse tridiagonal matrix. Compare nonzeros in $\mathbf{A}$ vs. LU factors.
2. **Timing:** Use `scipy.sparse.linalg.spsolve` vs. dense `np.linalg.solve` for $n = 2000$. Record memory and time.

5.2 Jacobi and Gauss-Seidel Methods

Standard stationary iterations based on diagonal and triangular splittings.

5.2.1 Jacobi Iteration

Decomposes $\mathbf{A}$ into diagonal $\mathbf{D}$ and remainder $\mathbf{R}$: $\mathbf{A} = \mathbf{D} + \mathbf{R}$.

Def 5.3 (Jacobi Scheme)

$$\mathbf{x}^{(k+1)} = \mathbf{D}^{-1}(\mathbf{b} - \mathbf{R}\mathbf{x}^{(k)})$$

- **Component form:** $x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$.
- **Concurrency:** All components update independently using values from the *previous* step.

Note 5.3 **Performance:** Jacobi is **embarrassingly parallel**. However, it converges slowly and requires storing both $\mathbf{x}^{(k)}$ and $\mathbf{x}^{(k+1)}$.

5.2.2 Gauss-Seidel Iteration

Splits $\mathbf{A} = \mathbf{L}_* + \mathbf{U}_*$, where $\mathbf{L}_*$ includes the diagonal.

Def 5.4 (Gauss-Seidel Scheme)

$$\mathbf{L}_* \mathbf{x}^{(k+1)} = \mathbf{b} - \mathbf{U}_* \mathbf{x}^{(k)}$$

- **Component form:** $x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right)$.
- **Key Idea:** Use updated values *immediately* within the same sweep.

Note 5.4 **Performance:** Sequential dependency prevents easy parallelization, but Gauss-Seidel typically converges **twice as fast** as Jacobi and updates $\mathbf{x}$ in-place.

5.2.3 Convergence Conditions

Thm 5.3 (Diagonal Dominance) If $\mathbf{A}$ is **strictly diagonally dominant** ($|a_{ii}| > \sum_{j \neq i} |a_{ij}|$), both Jacobi and Gauss-Seidel converge for any $\mathbf{x}^{(0)}$.

Thm 5.4 (SPD Convergence) If $\mathbf{A}$ is **SPD**, Gauss-Seidel is guaranteed to converge. Jacobi may still diverge.

Ex 5.3

1. For $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, compute one step of Jacobi and Gauss-Seidel from $\mathbf{0}$.
2. Compare $\rho(\mathbf{T}_J)$ and $\rho(\mathbf{T}_{GS})$. Does $\rho_{GS} \approx \rho_J^2$ hold?

5.2.4 Successive Over-Relaxation (SOR)

Accelerates Gauss-Seidel by overshooting the update.

Def 5.5 (SOR Update)

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \omega [\text{GS update}]$$

- **Convergence Condition:** $0 < \omega < 2$ (Kahan's Theorem).

Note 5.5 The “Square Root” Boost: For a 1D Poisson problem on a grid h :

- Jacobi/GS: $O(n^2)$ iterations.
- Optimal SOR: $O(n)$ iterations.

This turns a slow $O(h^2)$ method into a fast $O(h)$ method.

Ex 5.4

1. **Optimal Tuning:** For $\rho_J = 0.95$, calculate the optimal $\omega^* = 2/(1 + \sqrt{1 - \rho_J^2})$.
2. **Convergence Sweep:** Implement SOR and plot iterations to convergence vs. $\omega \in [1, 2)$. Identify the “cliff” at ω^* .

5.2.5 Stopping Criteria

1. **Absolute Residual:** $\|\mathbf{r}^{(k)}\| < \varepsilon$.
2. **Relative Residual:** $\|\mathbf{r}^{(k)}\|/\|\mathbf{b}\| < \varepsilon$. (Scale invariant).
3. **Increment:** $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| < \varepsilon$. (Cheap, but may stop prematurely if convergence is slow).

Ex 5.5

1. Why is a small residual not enough to guarantee a small error $\mathbf{e}^{(k)}$? (Condition number).
2. Implement a robust solver that combines relative residual with a `max_iter` budget.

5.3 Conjugate Gradient Method

Optimal Krylov subspace solver for symmetric positive definite (SPD) systems.

5.3.1 From Linear Systems to Optimization

Thm 5.5 (Quadratic Minimization) For SPD $\mathbf{A}$, solving $\mathbf{Ax} = \mathbf{b}$ is equivalent to minimizing the quadratic form:

$$\phi(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T \mathbf{Ax} - \mathbf{x}^T \mathbf{b}.$$

- **Gradient:** $\nabla\phi = \mathbf{Ax} - \mathbf{b} = -\mathbf{r}$.
- **Geometry:** ϕ is a paraboloid. $\kappa(\mathbf{A})$ determines the “narrowness” of the valley.

5.3.2 Conjugate Directions

Def 5.6 (**A-Conjugacy**) Nonzero vectors $\mathbf{p}, \mathbf{q}$ are **A-conjugate** if $\mathbf{p}^T \mathbf{A} \mathbf{q} = 0$.

- **Independence:** A set of **A-conjugate** vectors is linearly independent.
- **Optimality:** CG builds search directions that are **A-orthogonal** to all previous directions.

Note 5.6 **Performance:** Unlike steepest descent, which zigzags in narrow valleys, CG reaches the exact solution in at most n steps by searching each conjugate direction exactly once.

5.3.3 The CG Algorithm

Def 5.7 (**Conjugate Gradient Algorithm**) **Initialize:** $\mathbf{x}^{(0)} = \mathbf{0}, \mathbf{r}^{(0)} = \mathbf{b}, \mathbf{p}^{(0)} = \mathbf{r}^{(0)}$.
Iteration:

1. $\alpha_k = \frac{\mathbf{r}^{(k)T} \mathbf{r}^{(k)}}{\mathbf{p}^{(k)T} \mathbf{A} \mathbf{p}^{(k)}}$ (Step length)
2. $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}$ (Update iterate)
3. $\mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} - \alpha_k \mathbf{A} \mathbf{p}^{(k)}$ (Update residual)
4. $\beta_k = \frac{\mathbf{r}^{(k+1)T} \mathbf{r}^{(k+1)}}{\mathbf{r}^{(k)T} \mathbf{r}^{(k)}}$ (Gram-Schmidt weight)
5. $\mathbf{p}^{(k+1)} = \mathbf{r}^{(k+1)} + \beta_k \mathbf{p}^{(k)}$ (Next direction)

Note 5.7 **Efficiency:**

- **Matvecs:** Only 1 matrix-vector product per iteration.
- **Memory:** $O(n)$ storage (requires only a few vectors). No need to store the basis.

5.3.4 Convergence Rate

Thm 5.6 (**Convergence Bound**)

$$\frac{\|\mathbf{e}^{(k)}\|_{\mathbf{A}}}{\|\mathbf{e}^{(0)}\|_{\mathbf{A}}} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k.$$

Note 5.8 **The $\sqrt{\kappa}$ Advantage:** While stationary methods depend on $(\kappa - 1)/(\kappa + 1)$, CG depends on the **square root** of the condition number. For $\kappa = 100$, CG reduces error $5\times$ faster per step.

Note 5.9 **Superlinear Convergence:** If eigenvalues are clustered, CG converges much faster than the worst-case bound suggests (e.g., $k \ll n$ for clustered spectra).

5.3.5 Preconditioning

Replaces $\mathbf{Ax} = \mathbf{b}$ with $\mathbf{M}^{-1}\mathbf{Ax} = \mathbf{M}^{-1}\mathbf{b}$ where $\mathbf{M} \approx \mathbf{A}$.

Def 5.8 (Common Preconditioners)

1. **Jacobi:** $\mathbf{M} = \text{diag}(\mathbf{A})$. (Cheap, moderate speedup).
2. **Incomplete Cholesky (IC):** $\mathbf{M} \approx \mathbf{LL}^T$ with sparse factors. (Robust).
3. **Multigrid:** Near $O(n)$ total complexity for PDE systems.

Ex 5.6

1. **Convergence Check:** Solve a 100×100 Poisson system via CG. Compare your error plot to the theoretical bound.
2. **Spectral Clustering:** Construct $\mathbf{A}$ with eigenvalues $\{1, \dots, 1, 100, 200\}$. Observe superlinear convergence in the first few steps.
3. **Preconditioning:** Compare iteration counts for a sparse system with and without an IC preconditioner.

5.4 GMRES

Krylov subspace solver for general (non-symmetric) linear systems. Minimizes the residual norm at each step.

5.4.1 The Arnoldi Process

Constructs an orthonormal basis V_k for the Krylov subspace $\mathcal{K}_k(\mathbf{A}, \mathbf{r}_0)$.

Def 5.9 (Arnoldi Relation)

$$\mathbf{A}V_k = V_{k+1}\bar{H}_k$$

- V_k : Orthonormal columns.
- $\bar{H}_k$: Upper Hessenberg matrix $((k+1) \times k)$.
- **Insight:** $V_k^T \mathbf{A}V_k = H_k$ (projection of $\mathbf{A}$ onto the subspace).

5.4.2 GMRES as Least Squares

GMRES finds $\mathbf{x}_k \in \mathbf{x}_0 + \mathcal{K}_k$ such that $\|\mathbf{b} - \mathbf{A}\mathbf{x}_k\|_2$ is minimized.

Thm 5.7 (Least Squares Reduction)

$$\min \|\mathbf{b} - \mathbf{A}\mathbf{x}_k\|_2 = \min_{\mathbf{y} \in \mathbb{R}^k} \|\beta \mathbf{e}_1 - \bar{H}_k \mathbf{y}\|_2$$

where $\beta = \|\mathbf{r}_0\|_2$ and $\mathbf{e}_1 = (1, 0, \dots, 0)^T$.

Note 5.10 Optimality: By definition, GMRES finds the absolute best solution available in the growing Krylov subspace. The residual norm is non-increasing.

5.4.3 Restarted GMRES(m)

Note 5.11 The Memory Wall: Full GMRES storage costs $O(kn)$ and work grows quadratically with k (due to orthogonalization). **Restarting** after m steps prevents memory explosion but can lead to stagnation.

5.4.4 Convergence and Preconditioning

Thm 5.8 (Approximation Bound)

$$\frac{\|\mathbf{r}_k\|_2}{\|\mathbf{r}_0\|_2} \leq \min_{p \in \mathcal{P}_k, p(0)=1} \max_{\lambda \in \sigma(\mathbf{A})} |p(\lambda)|.$$

Def 5.10 (Preconditioning)

- **Left:** $\mathbf{M}^{-1}\mathbf{A}\mathbf{x} = \mathbf{M}^{-1}\mathbf{b}$. (Minimizes preconditioned residual).
- **Right:** $\mathbf{A}\mathbf{M}^{-1}(\mathbf{M}\mathbf{x}) = \mathbf{b}$. (Minimizes true residual; standard choice).

Ex 5.7

1. **GMRES vs. CG:** Compare residual plots for a symmetric system. Why is CG usually faster?
2. **Restarting:** Test GMRES(m) on a non-symmetric system for $m = 5, 20, 50$. Observe how small m can cause “plateaus” in convergence.
3. **Arnoldi:** Implement the Arnoldi process via Modified Gram-Schmidt. Verify the Hessenberg structure of $H_k = V_k^T \mathbf{A} V_k$.

6 Numerical Analysis

6.1 Numerical Quadrature

Approximating $I = \int_a^b f(x) dx$ via weighted sums: $\sum_{i=1}^n w_i f(x_i)$.

6.1.1 Newton-Cotes Rules

Uses equally-spaced nodes. Higher-order rules are prone to Runge's phenomenon.

Def 6.1 (Trapezoidal Rule) On n panels of width $h = (b - a)/n$:

$$T_n = h \left[\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(a + ih) \right].$$

Error Bound: $E = O(h^2)$. Doubling n reduces error $4\times$.

Def 6.2 (Simpson's Rule) Requires n to be even. Uses piecewise quadratic interpolation:

$$S_n = \frac{h}{3} \left[f(a) + f(b) + 4 \sum_{\text{odd } i} f(x_i) + 2 \sum_{\text{even } i} f(x_i) \right].$$

Error Bound: $E = O(h^4)$. Exact for polynomials of degree ≤ 3 .

6.1.2 Gaussian Quadrature

Optimizes **both** nodes and weights to achieve the highest possible degree of exactness.

Thm 6.1 (Gauss-Legendre Quadrature) Integrating over $[-1, 1]$ with p points:

$$\int_{-1}^1 f(x) dx \approx \sum_{i=1}^p w_i f(x_i).$$

Optimality: Exact for all polynomials of degree $\leq 2p - 1$. **Nodes:** Roots of the degree- p Legendre polynomial $P_p(x)$.

Note 6.1 Efficiency: p Gauss points are as accurate as $2p$ Newton-Cotes points. Always prefer Gaussian quadrature for smooth, non-periodic functions.

6.1.3 Richardson Extrapolation and Romberg Integration

Thm 6.2 (Richardson Extrapolation) Given an $O(h^2)$ estimate $A(h)$, build an $O(h^4)$ estimate by:

$$A_{\text{new}} = \frac{4A(h/2) - A(h)}{3}.$$

Def 6.3 (Romberg Integration) Iterative application of Richardson extrapolation to the trapezoidal rule. **Performance:** Error decreases as $O(h^{2k+2})$ for smooth f , converging spectacularly fast.

6.1.4 Adaptive Quadrature

Note 6.2 Design Goal: Automatically focus function evaluations where f is "difficult" (steep gradients, oscillations) and use large steps where f is smooth. **Mechanism:** Compare quadrature on one large panel vs. two small panels. If the difference is above tolerance ε , recurse.

6.1.5 Periodic Functions and Spectral Accuracy

Note 6.3 The Periodic Surprise: For smooth, periodic functions on $[0, 2\pi]$, the simple **Trapezoidal Rule** is the optimal choice. It achieves **Spectral Accuracy** (exponential convergence) because error terms in the Euler-Maclaurin expansion vanish at endpoints.

6.1.6 Exercises

Ex 6.1

1. Doubling nodes in Trapezoidal rule reduces error by $4\times$. How much does Simpson reduce it?
2. Derive the $p = 2$ Gauss points for $[-1, 1]$. (*Hint: roots of $3x^2 - 1$*).
3. Use Richardson extrapolation to improve $T(h)$ and $T(h/2)$ to get Simpson's rule.
4. Integration of $\sin(x)$ on $[0, 2\pi]$ with $n = 4$ nodes. Is the error 0? Why?

6.2 Root Finding

Iterative methods for solving $f(x) = 0$ for continuous $f : \mathbb{R} \rightarrow \mathbb{R}$.

6.2.1 Bracketing Methods

Guaranteed convergence to a root within a known interval.

Thm 6.3 (Bisection Method) Given $f \in C[a, b]$ with $f(a)f(b) < 0$, a root $x^* \in (a, b)$ exists. The method repeatedly halves the interval:

1. $m = (a + b)/2$
2. If $f(a)f(m) < 0$, new interval is $[a, m]$, else $[m, b]$.

Error Bound: After n steps, $|x_n - x^*| \leq (b - a)/2^n$.

Note 6.4 **Stability:** Bisection is the most robust method; it always converges if a bracket exists.
Convergence: ****Linear**** with rate $1/2$. Gains exactly 1 bit of accuracy per step.

6.2.2 Open Methods

Faster local convergence, but sensitive to the initial guess x_0 and may diverge.

Def 6.4 **(Newton's Method)** Linearizes f at the current iterate:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Thm 6.4 **(Quadratic Convergence)** If $f''(x)$ is continuous near a simple root x^* ($f'(x^*) \neq 0$), Newton's method converges ****quadratically****:

$$|x_{k+1} - x^*| \approx \frac{|f''(x^*)|}{2|f'(x^*)|} |x_k - x^*|^2.$$

Insight: The number of correct digits roughly doubles each step.

Note 6.5 **Failure Modes:** 1. x_0 too far from x^* . 2. $f'(x_k) \approx 0$ (zero derivative). 3. Multiple roots ($f'(x^*) = 0$): convergence degrades to linear with rate $1/2$.

Def 6.5 **(Secant Method)** Approximates f' via finite differences of the last two iterates:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

Requires two initial points (x_0, x_1) .

Thm 6.5 **(Superlinear Convergence)** The secant method converges with order $\phi = (1 + \sqrt{5})/2 \approx 1.618$ (the golden ratio). **Efficiency:** Faster than bisection, slower than Newton, but requires only 1 function evaluation per step (Newton requires 2: f and f').

6.2.3 Brent's Method

The standard robust choice for root-finding in 1D.

Thm 6.6 **(The Hybrid Approach)** Brent's method combines:

1. **Bisection:** For guaranteed global convergence.
2. **Secant:** For fast local convergence.
3. **IQI:** Inverse Quadratic Interpolation using 3 points.

Rule: It uses fast open steps by default, falling back to bisection only if the step is outside the bracket or not improving quickly enough.

Note 6.6 **Implementation:** Used in `scipy.optimize.brentq`.

6.2.4 Exercises

Ex 6.2

1. Bisection on $f(x) = x^3 - 2$ on $[1, 2]$ for 4 steps. Find the guaranteed error.
2. Newton on $f(x) = x^2 - 2$ from $x_0 = 1$. Verify the doubling of digits.
3. Prove that for $f(x) = x^2$, Newton converges to 0 with linear rate $1/2$.
4. Find the root of $e^x = 3x$ near $x = 1$ via Secant method. Compare evaluations vs. Newton.

7 Ordinary Differential Equations

7.1 Numerical Methods for ODEs

Numerical integration of the *initial value problem* (IVP):

$$\frac{dy}{dt} = f(t, y), \quad y(t_0) = y_0.$$

7.1.1 Basic Methods

Def 7.1 (Forward Euler (Explicit)) Advance via forward difference:

$$y_{n+1} = y_n + hf(t_n, y_n).$$

Error: Local $O(h^2)$, Global $O(h)$. **Stability:** Unstable for stiff problems. Absolute stability requires $h\lambda \in$ region of stability (circle in left half-plane).

Def 7.2 (Backward Euler (Implicit)) Advance via backward difference:

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1}).$$

Rule: Requires solving a nonlinear system at each step (via Newton). **Stability:** **A-Stable** (unconditionally stable for all h if $\text{Re}(\lambda) < 0$). The choice for stiff systems.

7.1.2 Runge-Kutta Methods

Multistage methods that match higher-order Taylor expansions without requiring analytic derivatives of f .

Def 7.3 (RK4 (Classical 4th Order)) Evaluates f at four stages per step:

$$\begin{aligned}k_1 &= f(t_n, y_n) \\k_2 &= f(t_n + h/2, y_n + hk_1/2) \\k_3 &= f(t_n + h/2, y_n + hk_2/2) \\k_4 &= f(t_n + h, y_n + hk_3) \\y_{n+1} &= y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)\end{aligned}$$

Error: Local $O(h^5)$, Global $O(h^4)$. Standard default for non-stiff systems.

Thm 7.1 (Adaptive Step Control (RK45)) Uses an **embedded pair** to estimate local error e without extra stages (Dormand-Prince). **Mechanism:** $h_{\text{new}} = h_{\text{old}}(\varepsilon/\|e\|)^{1/5}$. **Rule:** Automatically shrinks h in regions of high curvature and expands h where f is smooth.

7.1.3 Symplectic Integrators

Preserve the geometric structure of **Hamiltonian systems** (Energy conservation).

Def 7.4 (Velocity Verlet) For $H = \frac{1}{2}v^2 + V(q)$, ensures no long-term energy drift:

1. $v_{n+1/2} = v_n + \frac{h}{2}F_n$
2. $q_{n+1} = q_n + hv_{n+1/2}$
3. $v_{n+1} = v_{n+1/2} + \frac{h}{2}F_{n+1}$

Property: Symplectic and time-reversible. Numerical energy oscillates but stays bounded.

Note 7.1 The Drift Warning: Standard explicit methods like Forward Euler or RK4 exhibit monotonic energy growth/decay in Hamiltonian systems. Always use symplectic methods for long-term physics simulations.

7.1.4 Exercises

Ex 7.1

1. Solve $y' = -y, y(0) = 1$ with Forward Euler for 4 steps. Compare error to e^{-1} .
2. Verify the stability limit of Forward Euler for $y' = \lambda y$ is $h < 2/|\lambda|$.
3. Show that Forward Euler on a harmonic oscillator leads to unbounded energy growth.
4. Implement RK45 for $y' = \cos(t^2)$ and plot the step-size h over time.

8 Applications

8.1 Principal Component Analysis (PCA)

Optimal linear dimensionality reduction via variance maximization.

8.1.1 Definitions and Preprocessing

Def 8.1 (Data Centering) Data matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$ (n samples, p features) must be **centered**:

$$X_{ij} \leftarrow X_{ij} - \bar{x}_j, \quad \bar{x}_j = \frac{1}{n} \sum_{i=1}^n X_{ij}.$$

Rule: Throughout this section, $\mathbf{X}$ is assumed centered.

Def 8.2 (Sample Covariance Matrix) The matrix $\mathbf{S} = \frac{1}{n-1} \mathbf{X}^T \mathbf{X} \in \mathbb{R}^{p \times p}$.

- **Diagonal:** S_{jj} is the variance of feature j .
- **Off-diagonal:** S_{ij} is the covariance between features i and j .
- **Property:** $\mathbf{S}$ is symmetric positive semi-definite (SPSD).

8.1.2 PCA via SVD

Thm 8.1 (Principal Directions and Scores) Let $\mathbf{X} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$ be the SVD of the centered data matrix.

- **Principal Directions (Loadings):** Columns of $\mathbf{V}$ (eigenvectors of $\mathbf{S}$).
- **Principal Components (Scores):** $\mathbf{Z} = \mathbf{X} \mathbf{V} = \mathbf{U} \mathbf{\Sigma}$.
- **Variances:** $\lambda_i = \text{Var}(\mathbf{z}_i) = \frac{\sigma_i^2}{n-1}$.

Note 8.1 **Numerical Stability:** Never form $\mathbf{X}^T \mathbf{X}$ explicitly for PCA. Squaring the data matrix doubles the condition number ($\kappa(\mathbf{S}) = \kappa(\mathbf{X})^2$). Compute PCA directly via the SVD of $\mathbf{X}$.

8.1.3 Variance and Dimensionality Reduction

Def 8.3 (Proportion of Variance Explained (PVE)) For the first k components:

$$\text{PVE}_k = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^p \sigma_i^2}.$$

Rule: Choose k to reach a target PVE (e.g., 95%) or identify the "elbow" in a scree plot.

Thm 8.2 (Optimal Approximation) The rank- k matrix $\mathbf{X}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T$ is the optimal k -dimensional linear approximation of the data in the Frobenius norm (Eckart-Young).

8.1.4 Implementation Details

Note 8.2 **Standardization:** If features have different units (e.g., height in meters vs. weight in kg), PCA will be dominated by large-scale features. ****Standardize**** by dividing each centered column by its standard deviation: $X_{ij} \leftarrow (X_{ij} - \bar{x}_j)/s_j$.

Def 8.4 **(Kernel PCA (High-Dimensional Case))** When $p \gg n$, work with the $n \times n$ Gram matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^T$. Principal directions are recovered via $\mathbf{v}_i = \mathbf{X}^T \mathbf{u}_i / \sigma_i$.

8.1.5 Exercises

Ex 8.1

1. Center $\mathbf{X} = \begin{pmatrix} 2 & 1 \\ -1 & 3 \\ -1 & -4 \end{pmatrix}$ and compute $\mathbf{S}$. Find the PVE for $k = 1$.
2. Prove that PC scores $\mathbf{z}_i, \mathbf{z}_j$ are uncorrelated for $i \neq j$.
3. Load the Iris dataset. Plot the data in the basis of the first two PCs. Do species cluster?
4. Verify that the sum of variances $\sum \lambda_i$ equals the total variance $\text{Tr}(\mathbf{S})$.

8.2 Spectral Graph Theory and PageRank

Encoding graph topology into matrix spectrum for connectivity, clustering, and ranking.

8.2.1 Graph Matrices

Def 8.5 **(Adjacency and Degree Matrices)** For a graph $G = (V, E)$ with n vertices:

- **Adjacency A:** $A_{ij} = w_{ij}$ if $(i, j) \in E$, else 0.
- **Degree D:** Diagonal matrix with $D_{ii} = d_i = \sum_j w_{ij}$.

Def 8.6 **(Graph Laplacian)** Defined as $\mathbf{L} = \mathbf{D} - \mathbf{A}$.

- **Discrete Derivative:** $(\mathbf{L}\mathbf{f})_i = \sum_{j \sim i} w_{ij}(f_i - f_j)$.
- **Quadratic Form:** $\mathbf{x}^T \mathbf{L} \mathbf{x} = \sum_{(i,j) \in E} w_{ij}(x_i - x_j)^2$.

Thm 8.3 **(Spectral Properties)** For a connected graph with $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$:

1. $\lambda_1 = 0$ with eigenvector $\mathbf{1}$.

2. **Multiplicity of 0:** Equals the number of connected components.
3. **Fiedler Value (λ_2):** Measures algebraic connectivity. $\lambda_2 > 0$ iff graph is connected.

Def 8.7 (Normalized Laplacians)

- **Symmetric:** $\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2} \mathbf{L} \mathbf{D}^{-1/2}$. (Eigenvalues in $[0, 2]$).
- **Random Walk:** $\mathbf{L}_{\text{rw}} = \mathbf{D}^{-1} \mathbf{L} = \mathbf{I} - \mathbf{P}$.

8.2.2 Spectral Clustering

Thm 8.4 (Spectral Relaxation) Minimizing a graph cut (RatioCut) is NP-hard. Spectral methods relax this to an eigenvector problem:

1. Compute k smallest eigenvectors of $\mathbf{L}$ (starting from $\mathbf{v}_2$).
2. Embed vertices in $\mathbb{R}^k$ using these eigenvectors.
3. Cluster embedding via k -means.

8.2.3 Random Walks and PageRank

Def 8.8 (Random Walk Matrix) Transition matrix $\mathbf{P} = \mathbf{D}^{-1} \mathbf{A}$.

- $P_{ij} = w_{ij}/d_j$ is the probability of moving from j to i .
- **Stationary Distribution:** $\boldsymbol{\pi}$ such that $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$. For undirected graphs, $\pi_i = d_i / \sum d_j$.

Def 8.9 (PageRank and Google Matrix) Models a random surfer on a directed web graph with teleportation:

$$\mathbf{G} = d\mathbf{S} + \frac{1-d}{N}\mathbf{J}$$

- d is the damping factor (standard: 0.85).
- $\mathbf{S}$ is the link matrix (stochastic, handles dangling nodes).
- $\mathbf{J}$ is the all-ones matrix (teleportation).

Computation: PageRank vector $\mathbf{r}$ is the dominant eigenvector of $\mathbf{G}$, computed via power iteration.

8.2.4 Exercises

Ex 8.2

1. Construct $\mathbf{L}$ for a 3-vertex path graph. Find its eigenvalues by hand.
2. Show that $\mathbf{L}\mathbf{1} = \mathbf{0}$ for any graph.
3. Prove $\mathbf{L}$ is positive semi-definite using the quadratic form.
4. Implement PageRank power iteration for a 10-node random graph.

8.3 Linear Operators and the Poisson Equation

Transitioning from finite matrices to infinite-dimensional operators.

8.3.1 Differential Operators and Discretization

Def 8.10 (Linear Operator) Map $\mathcal{A} : X \rightarrow Y$ between function spaces satisfying:

$$\mathcal{A}(\alpha u + v) = \alpha \mathcal{A}u + \mathcal{A}v.$$

Matrices are finite-dimensional operators; $\frac{d}{dx}$ and ∇^2 are infinite-dimensional.

Def 8.11 (Finite Difference Stencil) Approximating $u''(x)$ on a grid with spacing h :

$$u''(x_k) \approx \frac{u(x_{k-1}) - 2u(x_k) + u(x_{k+1}))}{h^2}.$$

Accuracy: Second-order $O(h^2)$ via Taylor expansion.

Def 8.12 (1-D Discrete Laplacian) Discretizing $-d^2/dx^2$ on $[0, 1]$ with $u(0) = u(1) = 0$ yields the tridiagonal system $\mathbf{T}\mathbf{u} = \mathbf{f}$:

$$\mathbf{T} = \frac{1}{h^2} \text{tridiag}(-1, 2, -1) \in \mathbb{R}^{n \times n}.$$

Thm 8.5 (Spectrum Convergence) Eigenvalues of $\mathbf{T}$ are $\lambda_k = \frac{4}{h^2} \sin^2(\frac{k\pi}{2(n+1)})$. As $n \rightarrow \infty$, $\lambda_k \rightarrow k^2\pi^2$ (eigenvalues of continuous $-d^2/dx^2$).

8.3.2 Operator Norms and Conditioning

Def 8.13 (Boundedness) An operator is **bounded** if $\|\mathcal{A}u\|_Y \leq C\|u\|_X$ for some $C < \infty$.

- Matrices are always bounded.
- Differential operators are **unbounded**.

Note 8.3 Numerical Consequence: Because the continuous Laplacian is unbounded, its discretization $\mathbf{T}$ becomes increasingly ill-conditioned as $h \rightarrow 0$:

$$\kappa(\mathbf{T}) = \frac{\lambda_{\max}}{\lambda_{\min}} \approx \frac{4/h^2}{\pi^2} = O(h^{-2}).$$

Refining the grid $2\times$ quadruples the condition number.

8.3.3 Exercises

Ex 8.3

1. Build $\mathbf{T}$ for $n = 50$ and plot its eigenvalues vs. $k^2\pi^2$.
2. Show that $\kappa(\mathbf{T}) \rightarrow \infty$ as $n \rightarrow \infty$.
3. Implement a 2-D Poisson solver on a unit square. Verify $O(N^{3/2})$ cost for direct sparse solve.
4. Prove the differentiation operator is unbounded on $L^2[0, 1]$ using $u_k(x) = \sin(k\pi x)$.

8.4 Fourier Analysis as Change of Basis

Diagonalizing the Laplacian via sinusoidal eigenfunctions. Turn differential equations into algebra.

8.4.1 Discrete Fourier Transform (DFT)

Def 8.14 (DFT Matrix) Unitary matrix $\mathbf{F} \in \mathbb{C}^{n \times n}$ with entries:

$$F_{jk} = \frac{1}{\sqrt{n}} \omega_n^{jk}, \quad \omega_n = e^{-2\pi i/n}.$$

- **Unitary Property:** $\mathbf{F}^* \mathbf{F} = \mathbf{I}$. Orthonormal columns.
- **Inverse DFT:** $\mathbf{F}^{-1} = \mathbf{F}^*$.

Thm 8.6 (Parseval's Theorem) The DFT is an isometry: $\|\mathbf{F}\mathbf{x}\|_2 = \|\mathbf{x}\|_2$. Total energy is preserved between time and frequency domains.

8.4.2 Convolution and Circulant Matrices

Def 8.15 (Circulant Matrix) Matrix $\mathbf{C}$ where each row is a cyclic shift of the first column $\mathbf{c}$.

- **Diagonalization:** Every circulant matrix is diagonalized by the DFT: $\mathbf{C} = \mathbf{F}^* \text{diag}(\mathbf{F}\mathbf{c}) \mathbf{F}$.

- **Eigenvalues:** The eigenvalues of $\mathbf{C}$ are the DFT coefficients of its first column.

Thm 8.7 (Convolution Theorem) Circular convolution $\mathbf{z} = \mathbf{c} * \mathbf{x}$ is equivalent to pointwise multiplication in frequency:

$$\mathbf{F}(\mathbf{c} * \mathbf{x}) = (\mathbf{F}\mathbf{c}) \odot (\mathbf{F}\mathbf{x}).$$

Complexity: Reduced from $O(n^2)$ (direct) to $O(n \log n)$ (FFT).

8.4.3 The Fast Fourier Transform (FFT)

Thm 8.8 (Cooley-Tukey Algorithm) Computes the DFT in $O(n \log n)$ flops for $n = 2^p$.

- **Mechanism:** Recursively splits n -point DFT into two $n/2$ -point DFTs (even/odd indices).
- **NumPy:** Use `np.fft.fft` and `np.fft.ifft`.

Note 8.4 Spectral Differentiation: For smooth periodic signals, differentiate by multiplying frequency components $\hat{x}_k$ by ik . Provides exponential accuracy compared to polynomial accuracy of finite differences.

8.4.4 Exercises

Ex 8.4

1. Write out the 4×4 DFT matrix explicitly. Verify it is unitary.
2. Solve the periodic Poisson equation $-u'' = f$ via FFT. (*Hint: Laplacian is circulant*).
3. Compare the speed of `np.fft.fft(x)` vs. `F @ x` for $n = 4096$.
4. Implement spectral differentiation for $f(x) = \sin(x)$ and check error vs. n .

8.5 Markov Chains and Stochastic Matrices

Modeling random processes where future state depends only on the current state.

8.5.1 Definitions

Def 8.16 (Column-Stochastic Matrix) Matrix $\mathbf{P} \in \mathbb{R}^{n \times n}$ where:

1. $P_{ij} \geq 0$.
2. $\sum_i P_{ij} = 1$ (Columns sum to 1).

Property: $\lambda = 1$ is always an eigenvalue (dominant).

Def 8.17 (Markov Chain) Probability distributions $\mathbf{x}^{(k)}$ evolving by $\mathbf{x}^{(k+1)} = \mathbf{P}\mathbf{x}^{(k)}$.

- After k steps: $\mathbf{x}^{(k)} = \mathbf{P}^k \mathbf{x}^{(0)}$.
- Long-run behavior is governed by the eigendecomposition of $\mathbf{P}$.

Def 8.18 (Stationary Distribution) A probability vector $\boldsymbol{\pi}$ such that $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$.

- Right eigenvector of $\mathbf{P}$ for $\lambda = 1$.
- Represents the equilibrium state of the chain.

8.5.2 Convergence and Mixing

Thm 8.9 (Perron-Frobenius for Chains) If $\mathbf{P}$ is **irreducible** (all states reachable) and **primitive** (no cycles), then:

1. $\lambda_1 = 1$ is simple and unique.
2. $|\lambda_i| < 1$ for all other eigenvalues.
3. $\lim_{k \rightarrow \infty} \mathbf{P}^k \mathbf{x}^{(0)} = \boldsymbol{\pi}$ for any initial state.

Def 8.19 (Spectral Gap and Mixing Time) The gap $\delta = 1 - |\lambda_2|$ controls convergence speed.

- **Mixing Time:** $k_\epsilon \approx \log(1/\epsilon)/\delta$.
- **Insight:** Large gap = fast mixing. Connected graphs have larger gaps.

8.5.3 Properties and Design

Def 8.20 (Detailed Balance (Reversibility)) A chain is reversible if $\pi_j P_{ij} = \pi_i P_{ji}$ for all i, j .

- **Implication:** Reversible chains have real eigenvalues.
- **Symmetry:** $\mathbf{D}_\pi^{-1/2} \mathbf{P} \mathbf{D}_\pi^{1/2}$ is symmetric.

Note 8.5 Design Tool: Metropolis-Hastings. Construct a reversible chain for a desired $\boldsymbol{\pi}$ by accepting moves with probability $a = \min(1, \frac{\pi_{\text{new}} Q(\text{old}|\text{new})}{\pi_{\text{old}} Q(\text{new}|\text{old})})$.

8.5.4 Exercises

Ex 8.5

1. Construct $\mathbf{P}$ for a 2-state chain with transition probabilities $\{0.1, 0.2\}$. Find $\boldsymbol{\pi}$.
2. Prove that $|\lambda| \leq 1$ for any stochastic matrix.
3. Simulate 100 steps of PageRank on a 5-node graph. Check convergence rate vs. λ_2 .
4. Find $\boldsymbol{\pi}$ for a random walk on K_4 (complete graph).

8.6 Koopman Theory, DMD, and SINDy

Data-driven discovery of linear models for nonlinear dynamics.

8.6.1 Koopman Operator Theory

Def 8.21 (Nonlinear Dynamics and Observables) $\mathbf{x}^{(k+1)} = \mathbf{F}(\mathbf{x}^{(k)})$ on $\mathcal{M} \subseteq \mathbb{R}^n$.

- **Observable:** Scalar function $g : \mathcal{M} \rightarrow \mathbb{R}$.
- **Koopman Operator $\mathcal{K}$:** Linear operator acting on observables: $(\mathcal{K}g)(\mathbf{x}) = g(\mathbf{F}(\mathbf{x}))$.

Lifting: $\mathcal{K}$ is linear even if $\mathbf{F}$ is nonlinear, at the cost of being infinite-dimensional.

Def 8.22 (Koopman Eigenfunctions) Observable φ satisfying $\mathcal{K}\varphi = \mu\varphi$.

- **Temporal Evolution:** $\varphi(\mathbf{x}^{(k)}) = \mu^k \varphi(\mathbf{x}^{(0)})$.
- **Significance:** Provides a global linear coordinate system for nonlinear dynamics.

8.6.2 Dynamic Mode Decomposition (DMD)

Def 8.23 (DMD Algorithm) Finds the best linear fit $\mathbf{Y} \approx \mathbf{A}\mathbf{X}$ for snapshot matrices:

$$\mathbf{X} = [\mathbf{x}^{(0)} \dots \mathbf{x}^{(m-1)}], \quad \mathbf{Y} = [\mathbf{x}^{(1)} \dots \mathbf{x}^{(m)}].$$

1. Compute truncated SVD: $\mathbf{X} \approx \mathbf{U}_r \boldsymbol{\Sigma}_r \mathbf{V}_r^T$.
2. Construct $r \times r$ reduced operator: $\tilde{\mathbf{A}} = \mathbf{U}_r^T \mathbf{Y} \mathbf{V}_r \boldsymbol{\Sigma}_r^{-1}$.
3. DMD eigenvalues $\{\lambda_i\}$ and modes $\{\boldsymbol{\phi}_i\}$ are recovered from $\tilde{\mathbf{A}}$.

Note 8.6 **Interpretation:** DMD decomposes data into spatiotemporal modes $\boldsymbol{\phi}_i$ with associated frequencies and growth rates. It is the data-driven approximation of the Koopman operator using the identity dictionary.

8.6.3 SINDy: Sparse Identification

Def 8.24 (Sparse Regression for Dynamics) Identifies governing equations $\dot{\mathbf{x}} = \Xi^T \Theta(\mathbf{x})$ where Θ is a library of candidate functions (monomials, sin, etc.).

- **STLS Algorithm:** Sequential Thresholded Least Squares. Iteratively solves least squares and thresholds small coefficients to promote sparsity.
- **Outcome:** Parsimonious, interpretable models (e.g., recovering Lorenz equations).

8.6.4 Exercises

Ex 8.6

1. Prove the linearity of the Koopman operator.
2. Implement DMD for a 2D damped oscillator. Verify the identified eigenvalues.
3. Build a polynomial library Θ for $\mathbf{x} \in \mathbb{R}^2$ up to degree 2.
4. Use SINDy (via STLS) to identify the Lotka-Volterra predator-prey model from noisy data.